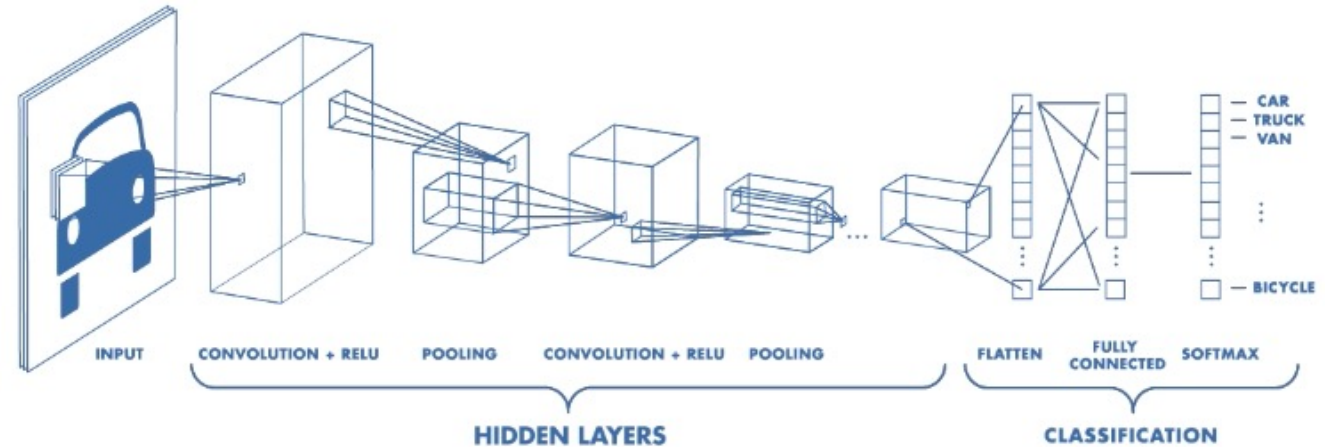


COMP9517

Computer Vision

2026 Term 1 Week 7

Dr Sonit Singh



Deep Learning

Image Classification using CNNs

Introduction

Image Classification

- Image classification with linear classifier

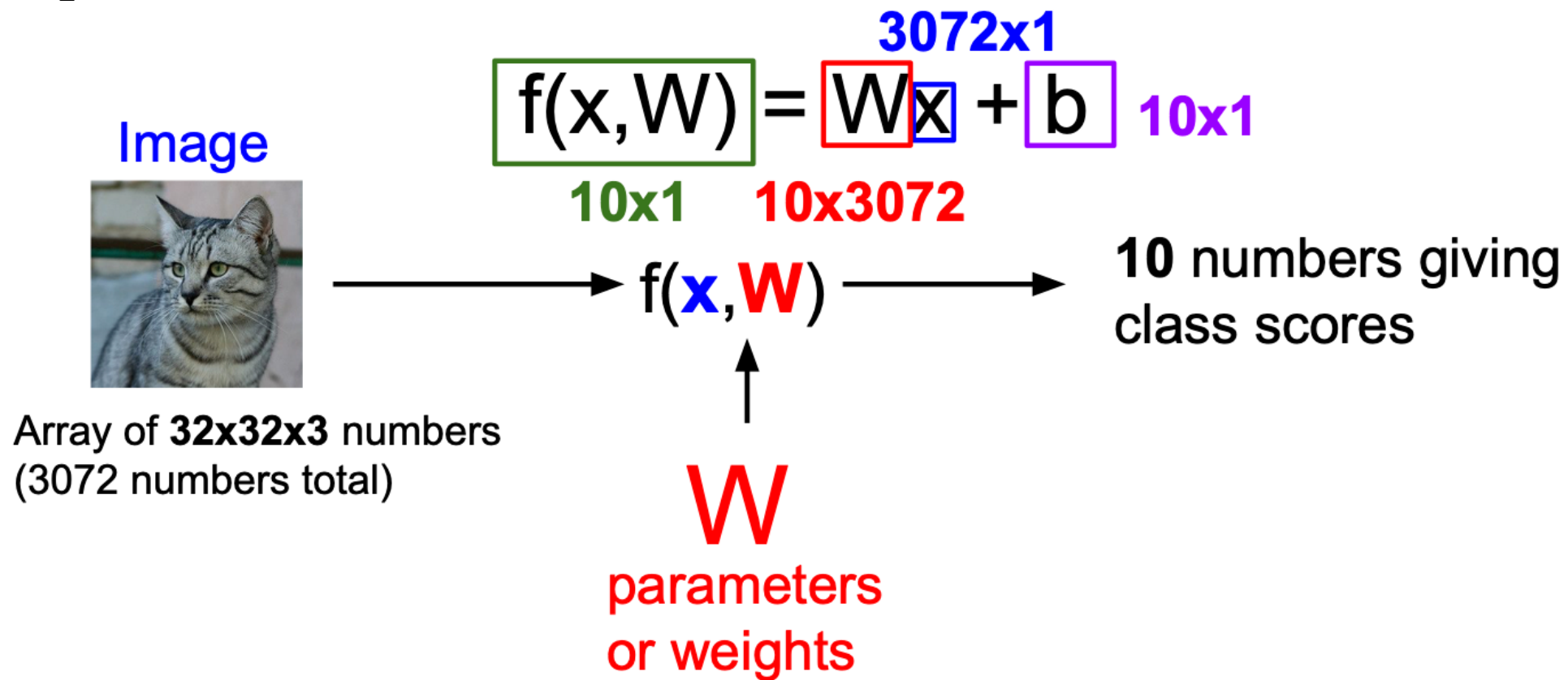
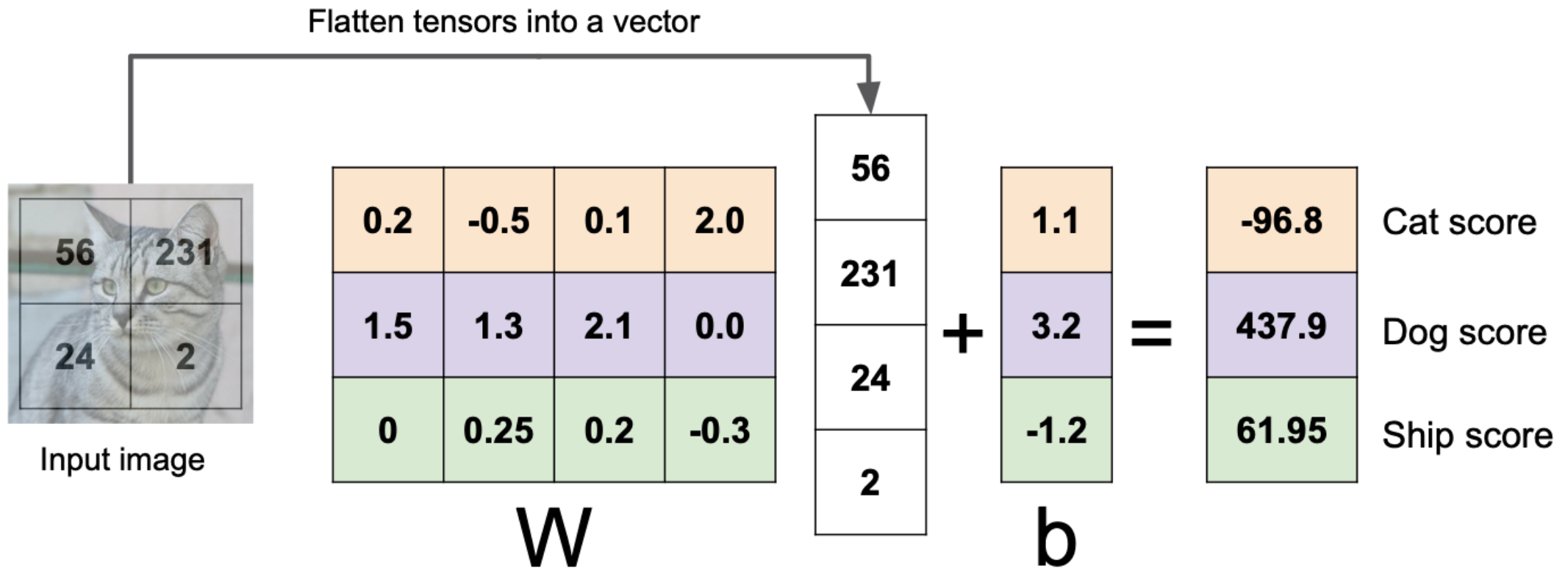


Image Classification

- An image example with 4 pixels and 3 classes.



Role of CNNs in Image Classification

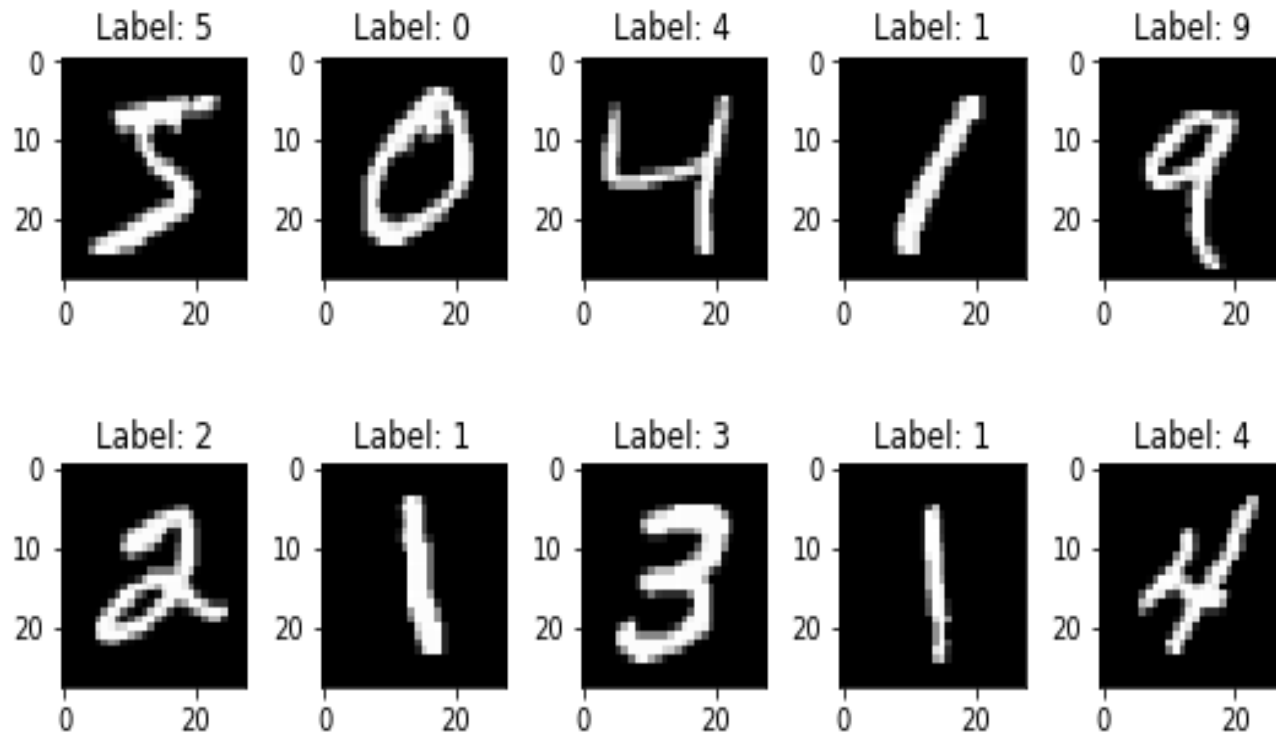
- CNNs are **designed specifically for processing grid-like data**, making them well-suited for images.
- They **can automatically learn relevant features from raw pixel data**, eliminating the need for handcrafted feature engineering.
- CNNs **use convolutional layers to detect local patterns, followed by pooling layers to reduce spatial dimensions** and prevent overfitting.
- The **fully connected layers at the end of a CNN make the final classification decision based on the learned features**.

Advantages of CNNs in Image Classification

- Automatic Feature Extraction
- Hierarchical Feature Learning
- Weight Sharing and Parameter Efficiency
- Transfer Learning and Pretrained Model
- Translation Invariance
- Superior Performance
- Robustness to Variations
- Scalability

Datasets

MNIST



MNIST

Overview:

- MNIST, short for "Modified National Institute of Standards and Technology," is a widely used dataset in the field of machine learning and computer vision.
- It consists of a collection of handwritten digits that are extensively used for tasks such as digit recognition and image classification.

Key Characteristics:

- 70,000 grayscale images.
- 28x28 pixels in size.
- Each image contains a single digit (0 through 9).
- Labeled dataset: Each image is associated with a corresponding digit label.

CIFAR-10

CIFAR10 [Alex Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”, Technical Report, 2009.]

airplane

automobile

bird

cat

deer

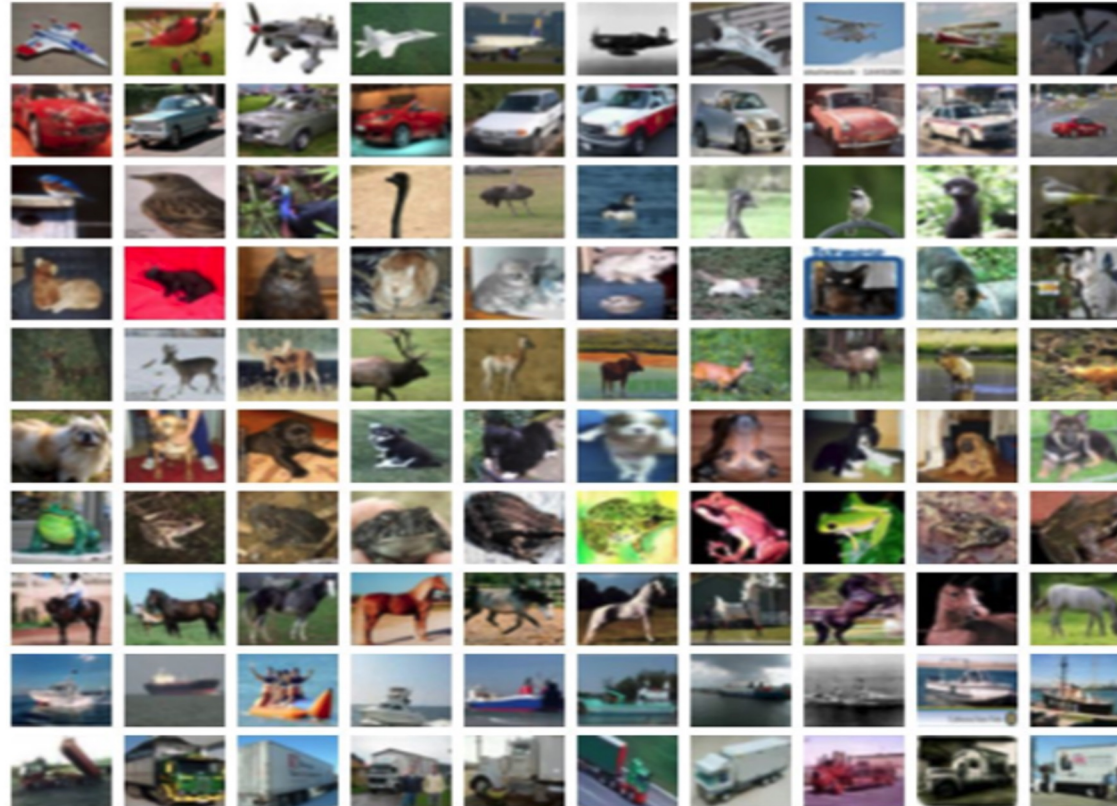
dog

frog

horse

ship

truck



10 classes

50,000 training images

10,000 testing images

CIFAR-10

Overview:

- CIFAR-10 stands for the “Canadian Institute For Advanced Research - 10,” and it is a widely used dataset for image classification tasks in the field of machine learning and computer vision.

Key Characteristics:

- CIFAR-10 consists of 60,000 color images.
- These images are divided into 10 different classes, each representing a distinct object or category.
- The dataset is split into 50,000 training images and 10,000 testing images.
- Each image is 32x32 pixels in size.
- The 10 classes include objects like airplanes, automobiles, birds, cats, and more.
- CIFAR-100 version

ImageNet

Consists of 14 million images, more than 21,000 classes, and about 1 million images have bounding box annotations

- Annotated by humans using crowdsourcing platform “Amazon Mechanical Turk”



- ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)
- annual competition to foster the development and benchmarking of state-of-the-art algorithms in Computer Vision
- Led to improvement in architectures and techniques at the intersection of CV and DL

Image Credit: Synced. <https://syncedreview.com/2020/06/23/google-deepmind-researchers-revamp-imagenet/>

COCO

Common Objects in Context (COCO) dataset is a large-scale object detection, segmentation, and captioning dataset.

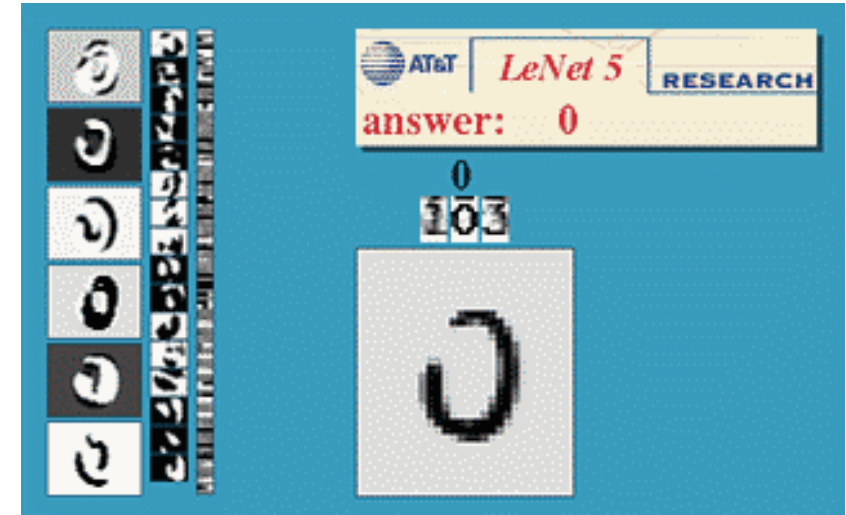
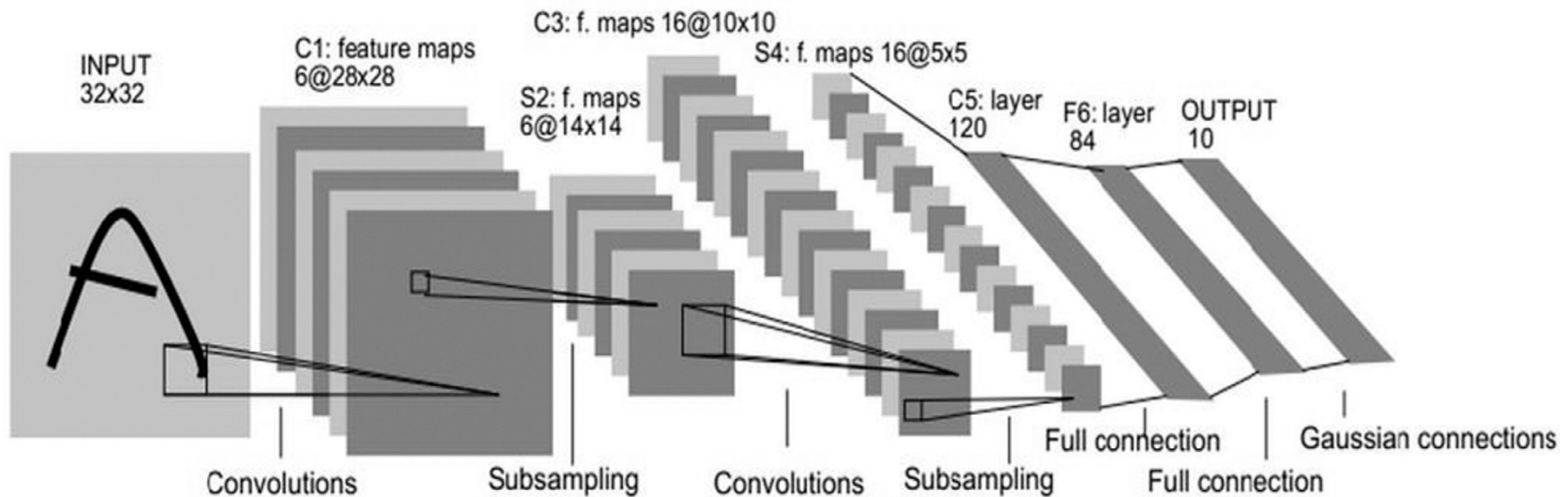


➤ <https://cocodataset.org/#home>

CNN Architectures

LeNet-5

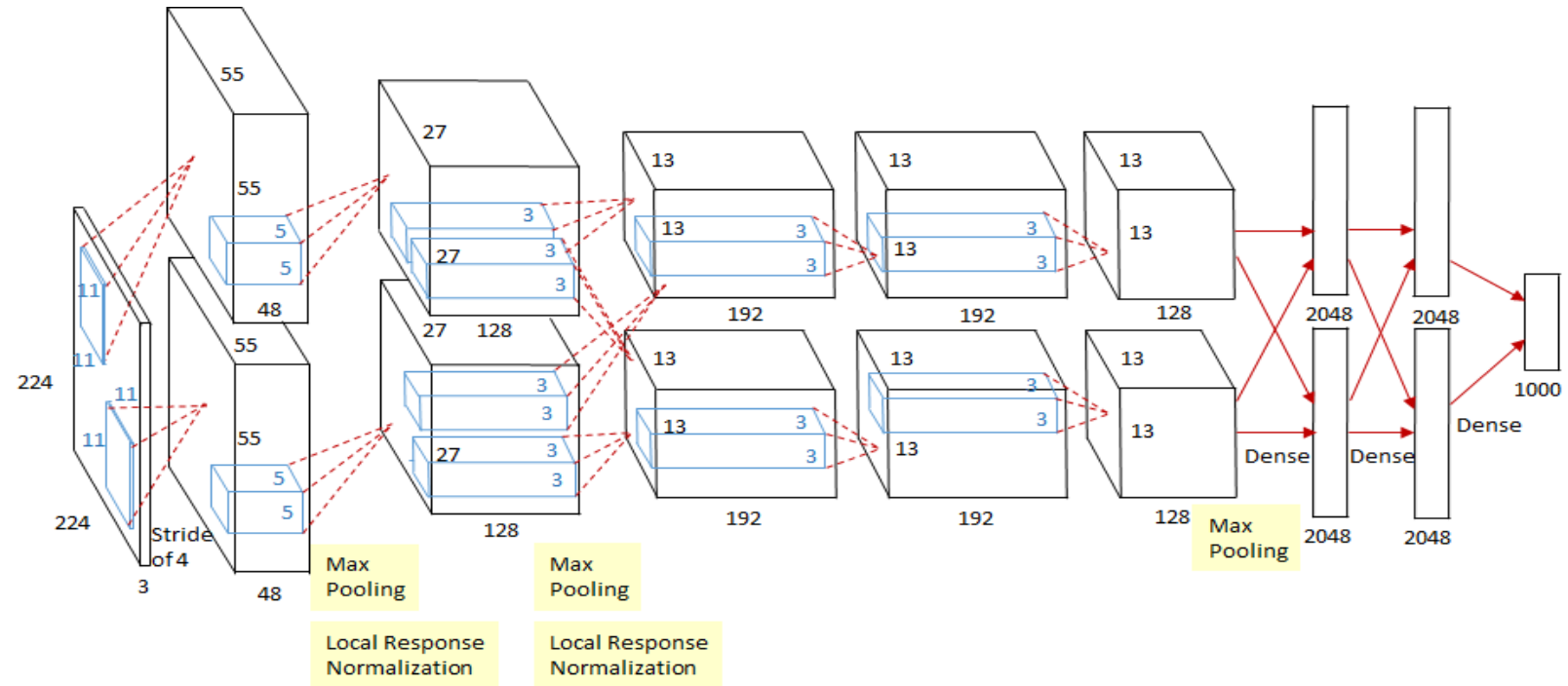
- First developed by Yann Lecun in 1989 for digit recognition
 - First time backprop is used to automatically learn visual features
 - Two convolutional layers, three fully connected layers (32 x 32 input, 6 and 12 FMs, 5 x 5 filters)
 - Stride = 2 is used to reduce image dimensions
 - Scaled tanh activation function
 - Uniform random weight initialization



Source: Lecun et al. (1989). Gradient-based learning applied to document recognition.

AlexNet

- Added super important heuristics
 - ReLU non-linearity
 - Local Response Normalization
 - Data Augmentation
 - Dropout
- **Winner of 2012 ILSVRC**

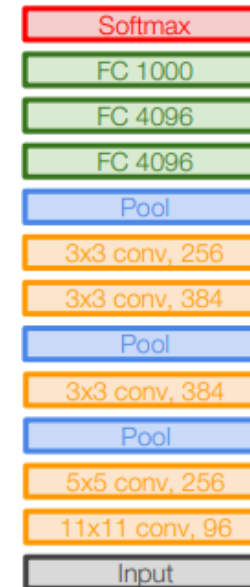


ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
<https://www.image-net.org/challenges/LSVRC/>

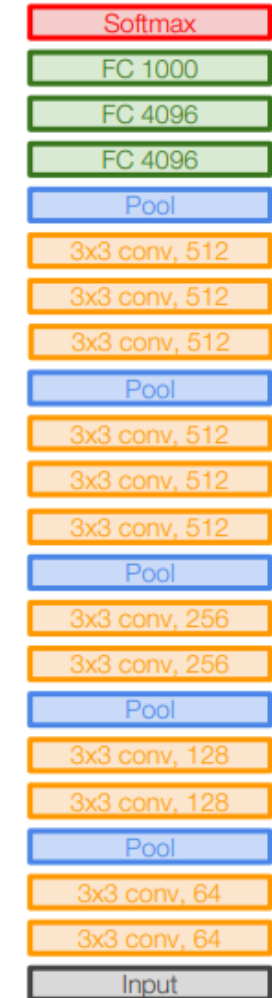
Image Credit: Kdnuggets – Deep Learning's Most Important Ideas. <https://www.kdnuggets.com/2020/09/deep-learning-most-important-ideas.html>

VGG

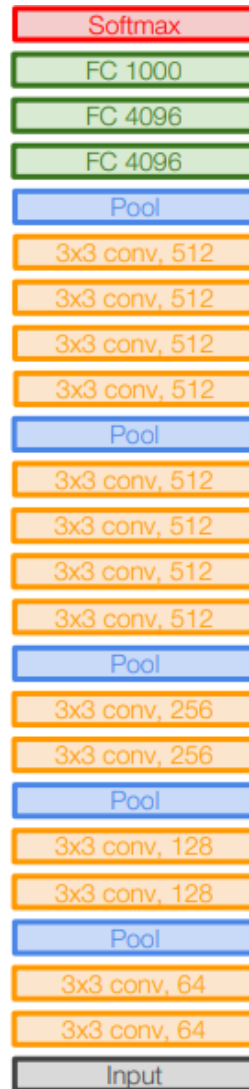
- Developed at Visual Geometry Group (Oxford) by Simonyan and Zisserman
- 1st runner up (Classification) and Winner (localization) of ILSVRC 2014 competition
- VGG-19 comprises of 144 million parameters



AlexNet



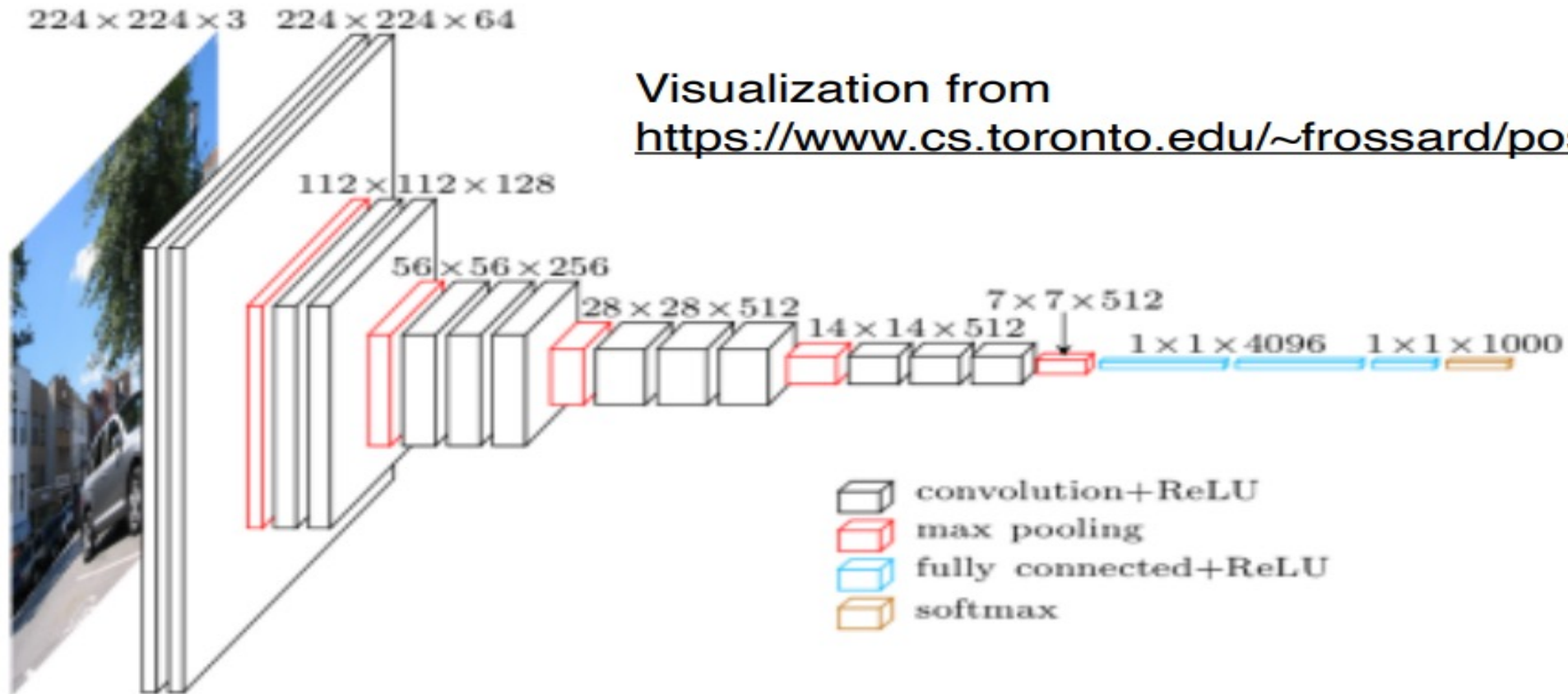
VGG16



VGG19

Image Credit: Medium. <https://medium.com/coinmonks/paper-review-of-vggnet-1st-runner-up-of-ilsvlc-2014-image-classification-d02355543a11>

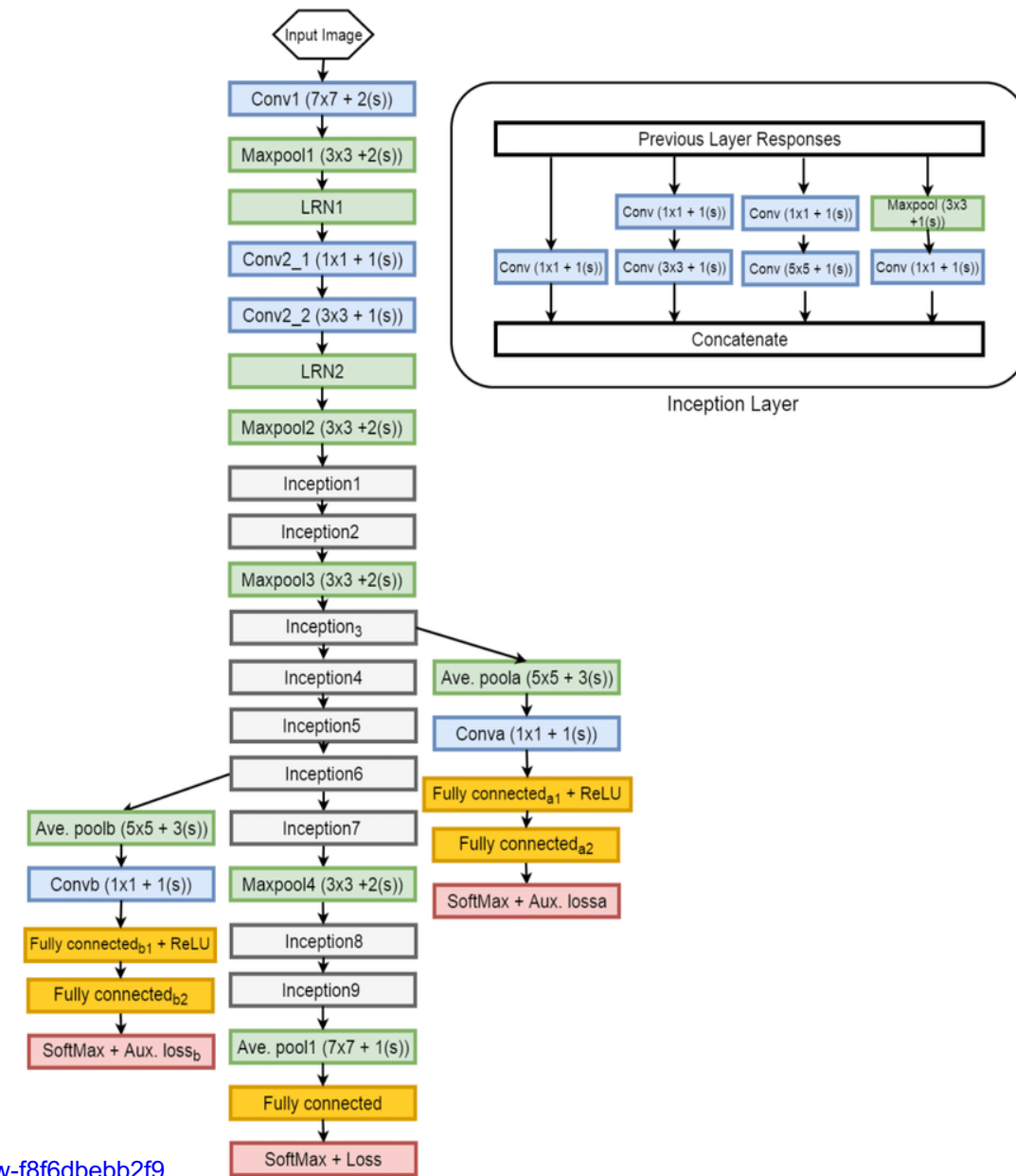
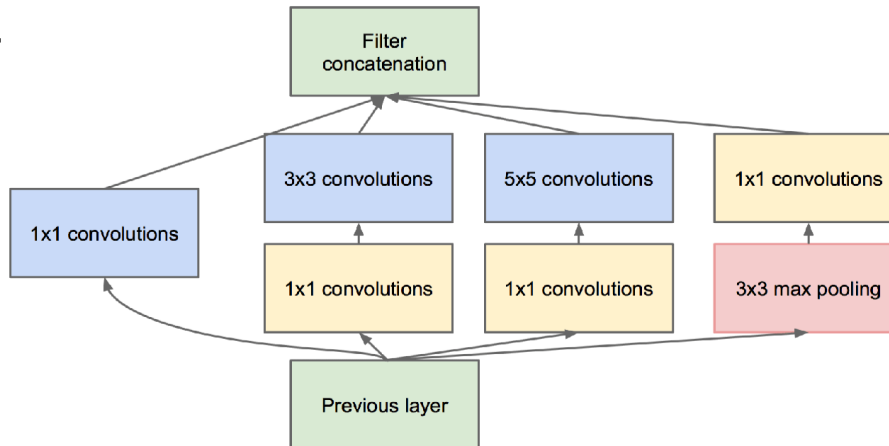
VGG-16



Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).

GoogLeNet

- A 22-layer CNN developed by researchers at Google
- Deeper networks prone to overfitting and suffer from **exploding or vanishing gradient problem**
- Core idea “**Inception module**”
 - Multi-branch, Multi-size kernel
- Adding **Auxiliary loss** as an extra supervision
- Winner of 2014



Source: Convolutional Neural Networks. <https://medium.com/@rajat.k.91/convolutional-neural-networks-why-what-and-how-f8f6dbebb2f9>

ResNet

- Developed by researchers at Microsoft (Kaiming et al.)
- Core idea “**residual connections**” or “**skip connections**” to preserve the gradient
- The **identity matrix** transmits forward the input data that **avoids the loss of information** (the data vanishing problem)

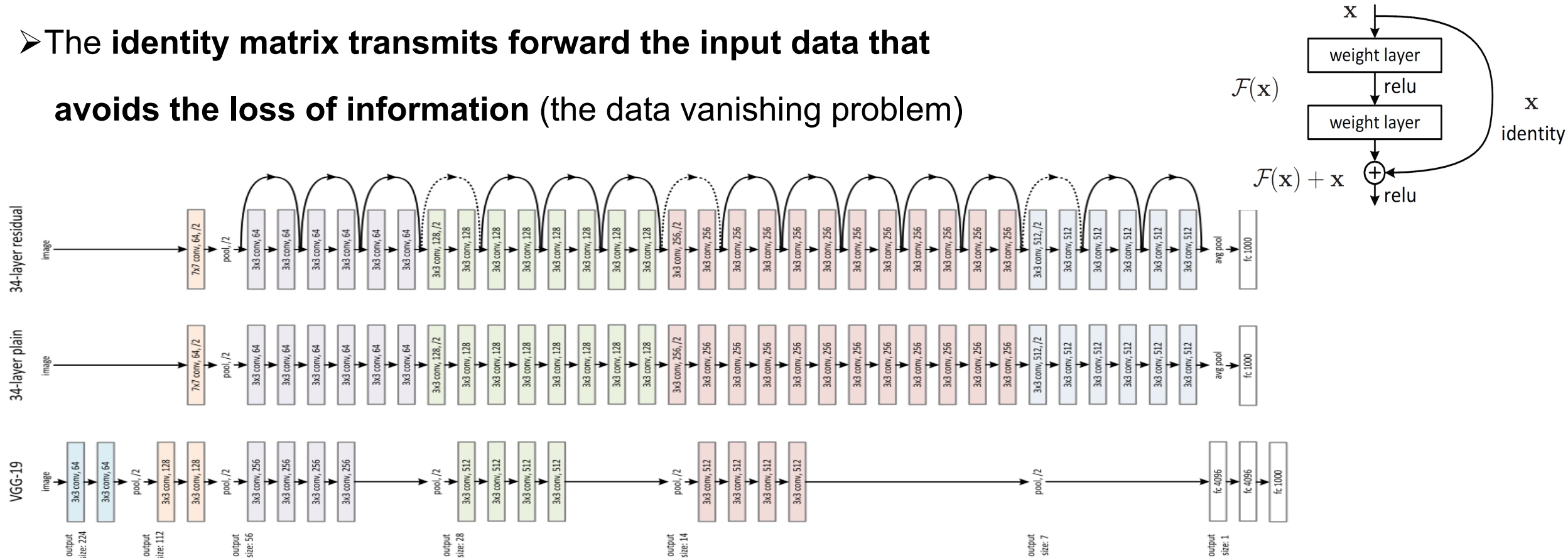
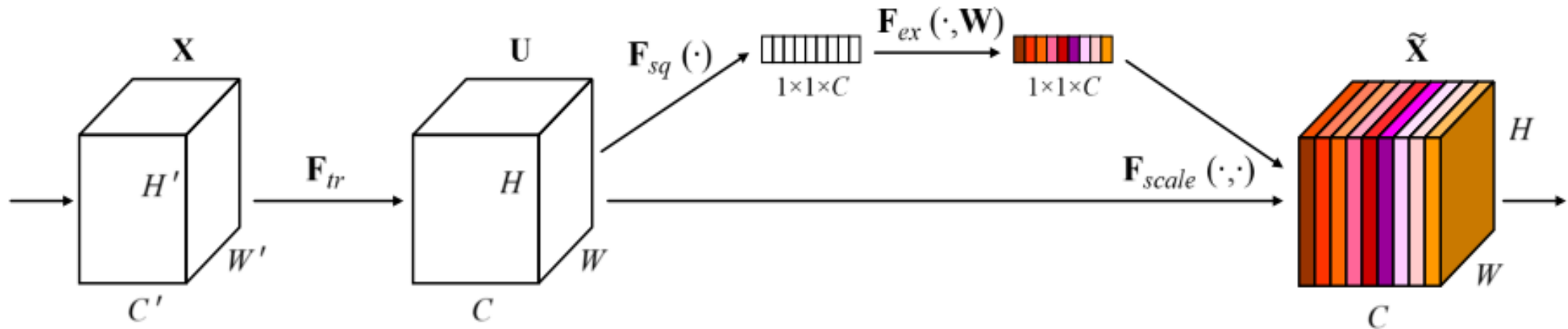


Image Credit: Medium. https://medium.com/@pierre_guillou/understand-how-works-resnet-without-talking-about-residual-64698f157e0c

SENet (Squeeze-and-Excitation Network)

- CNNs fuse the spatial and channel information to extract features to solve the task
- Before this, networks weights each of its channels equally when creating the output feature maps
- SENets added a content aware mechanism to weight each channel adaptively
- SE block helps to improve representation power of the network, able to better map the channel dependency along with access to global information

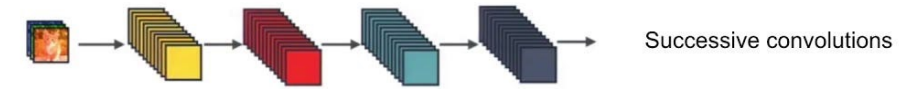


Source: Convolutional Neural Networks. <https://medium.com/@rajat.k.91/convolutional-neural-networks-why-what-and-how-f8f6dbebb2f9>

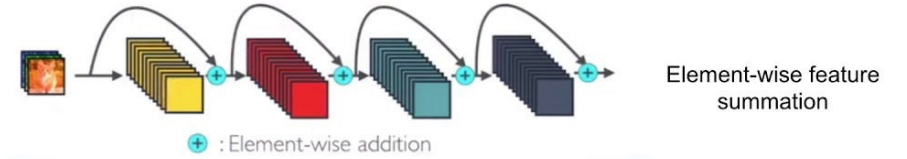
DenseNet

- “Dense block” vs “Res block”
- More flexible connections
- Transition layer is used between dense blocks
 - Reduce dimensionality and computation

Standard Connectivity



Resnet Connectivity



DenseNet Connectivity

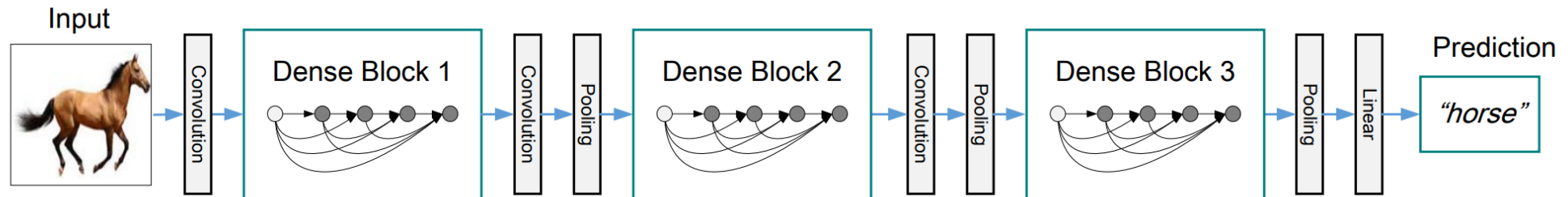
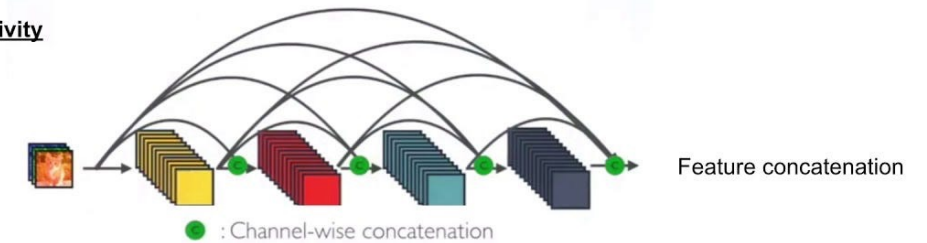


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Huang, Gao, et al. "Densely connected convolutional networks." *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 2017.

EfficientNet

- Efficient utilizes a **compound scaling method to uniformly scale depth, width, and resolution, providing high accuracy with computational efficiency.**
- Instead of randomly scaling up width, depth or resolution, compound scaling uniformly scales each dimension with a certain fixed set of scaling coefficients.
- **If the input image is bigger, then the network needs more layers to increase the receptive field and more channels to capture more fine-grained patterns on the bigger image.**

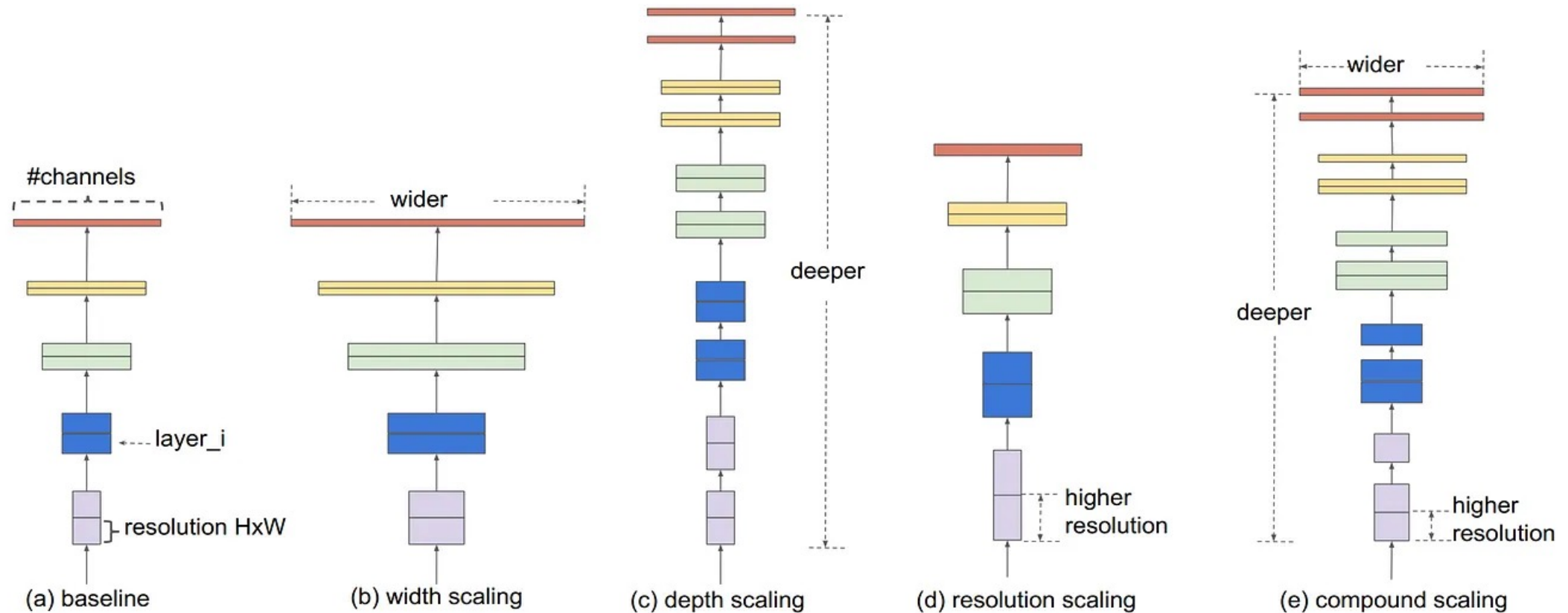
$$\text{Depth } d = \alpha^\phi, \text{ Width } w = \beta^\phi, \text{ Resolution } r = \gamma^\phi, (1)$$

$$\text{such that } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Tan, et al. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks " *ICML 2019*.

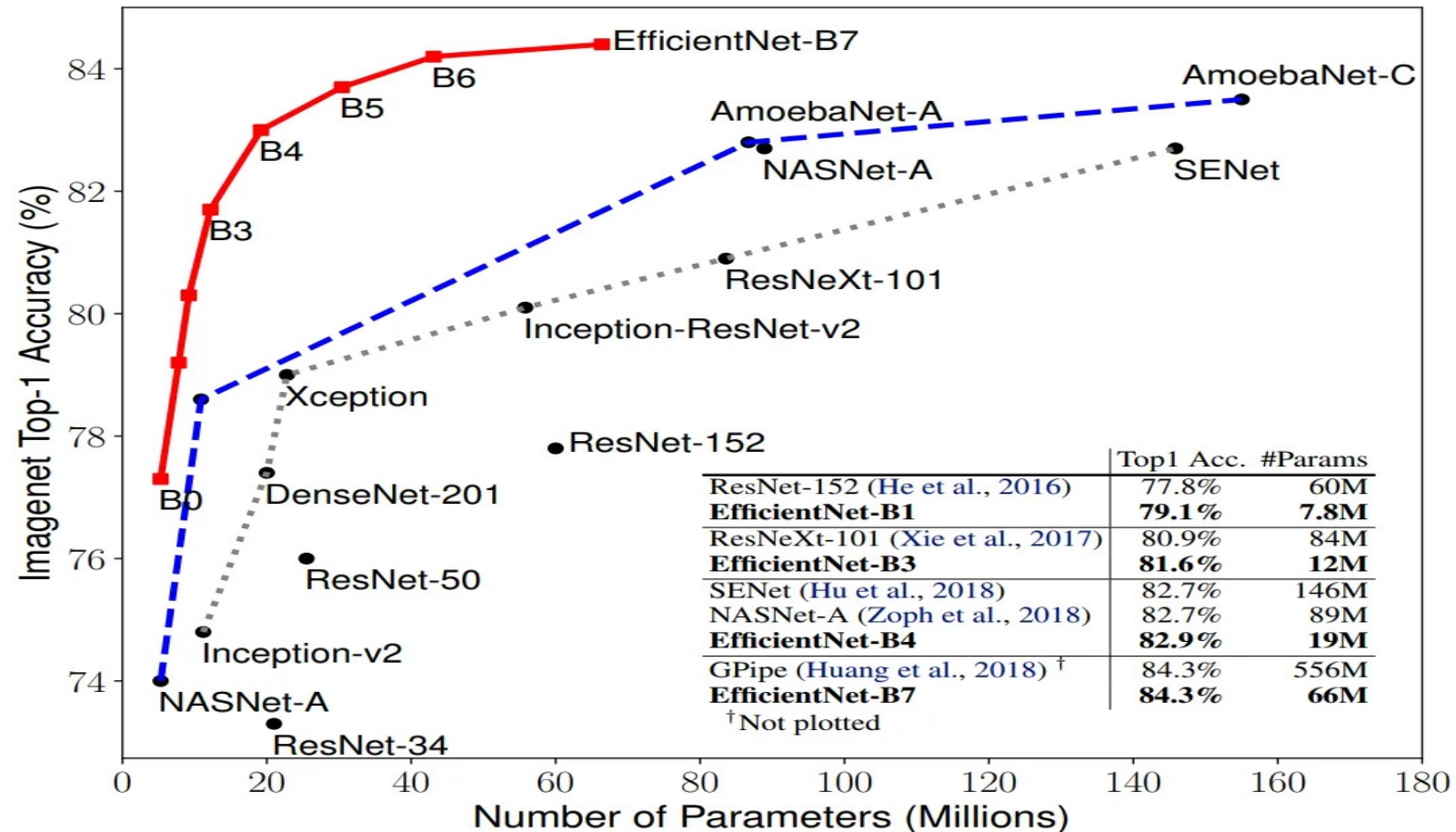
EfficientNet



Tan, et al. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks " *ICML 2019*.

EfficientNet

- EfficientNet size and performance on the ImageNet dataset

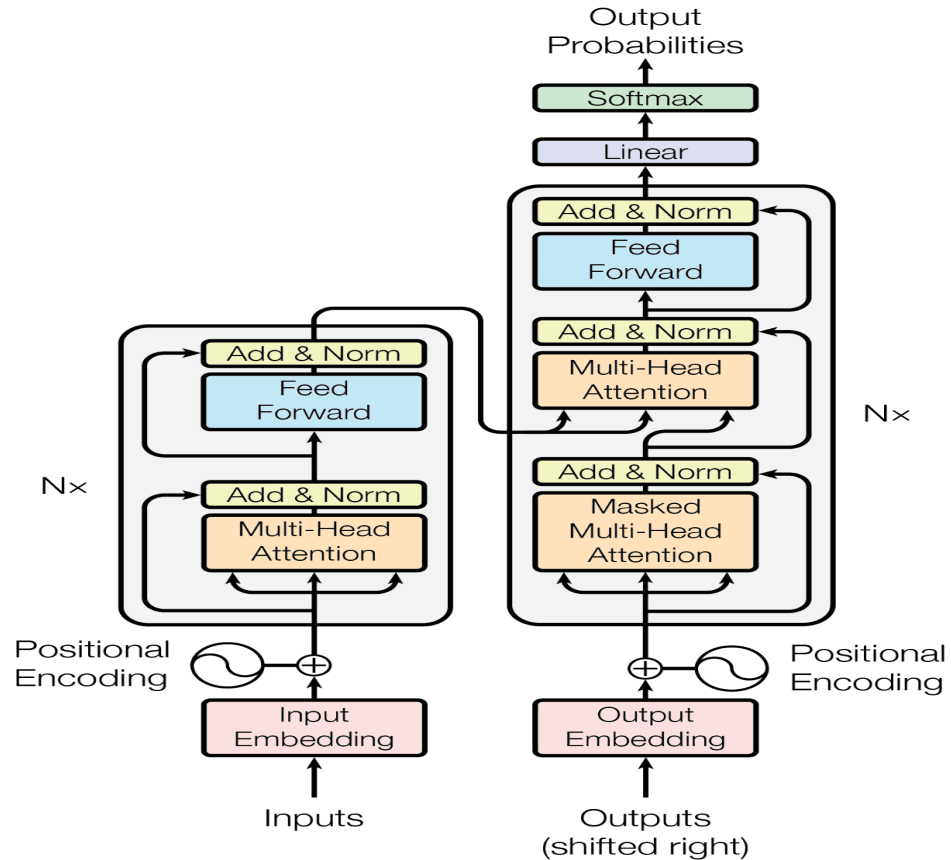


Tan, et al. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks" *ICML 2019*.

Vision Transformer (ViT)

- First application of Transformer model to Images

Vision Transformers

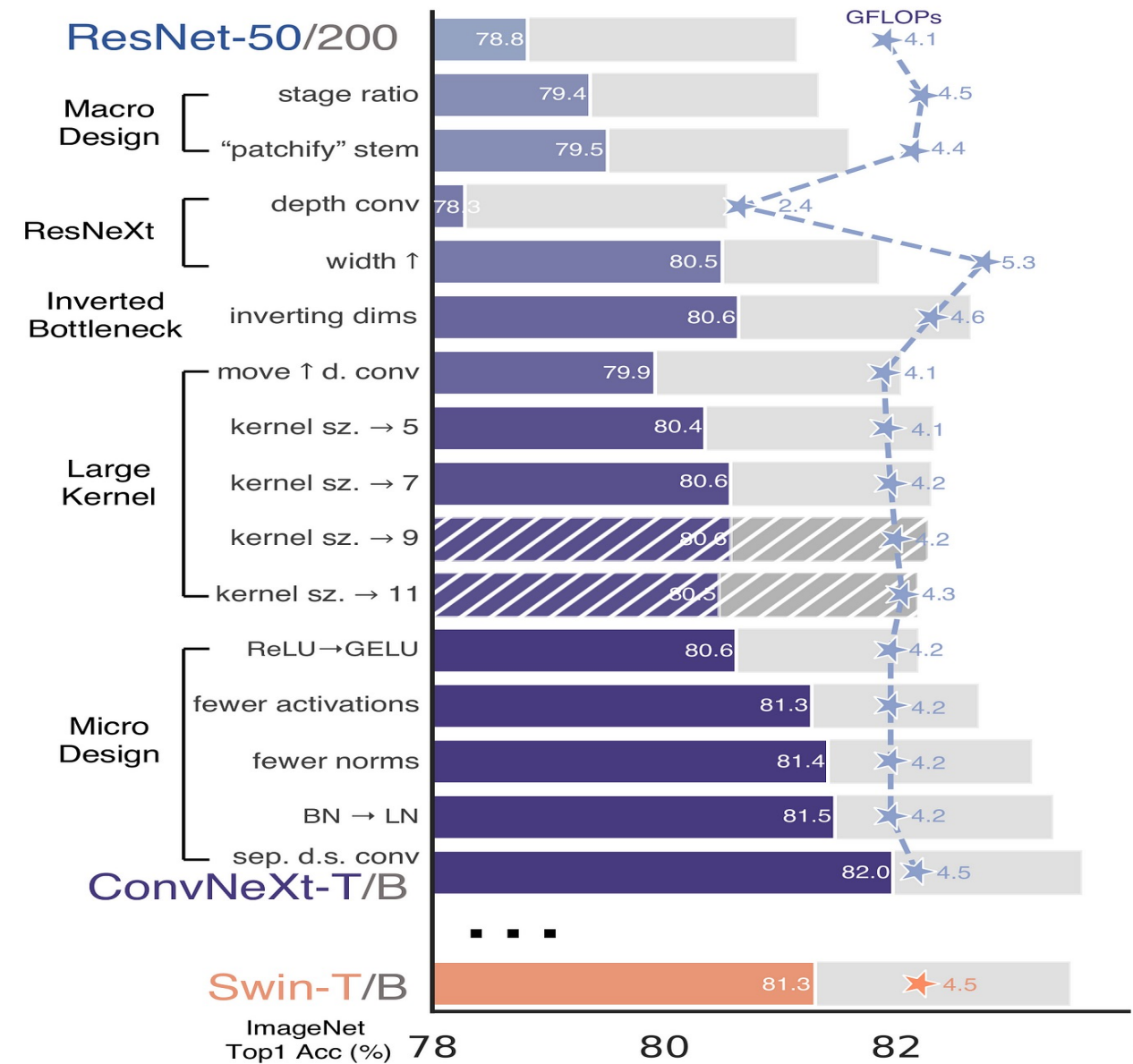


Transformers | Davide Cioconini | 2021

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.

ConvNeXt

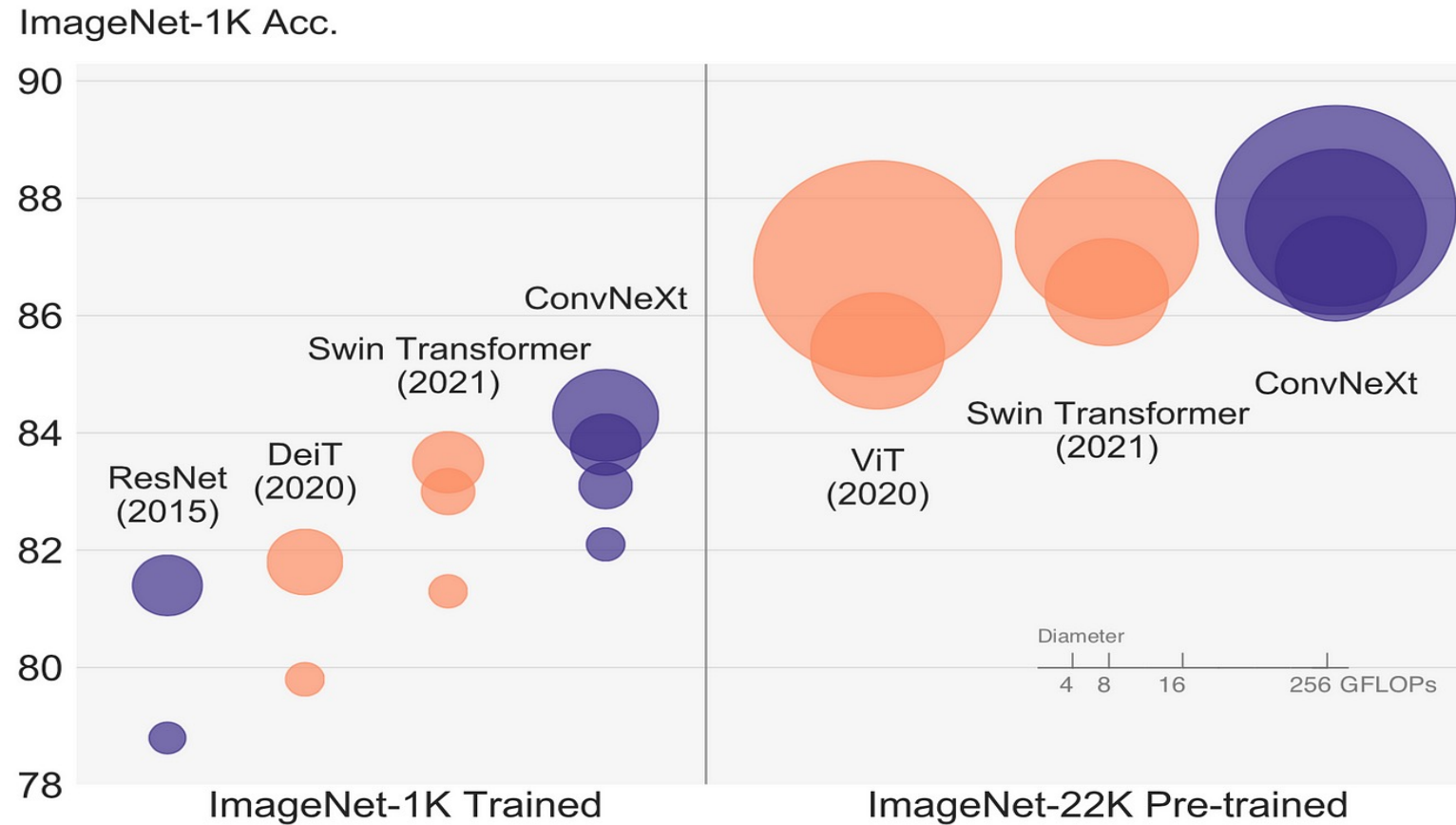
- A ConvNet for the 2020s
- With the rise of Transformer model, shift towards ViT and SwinTransformer
- Introduced image-specific inductive biases



Liu et al. "A ConvNet for the 2020s" CVPR 2022. https://openaccess.thecvf.com/content/CVPR2022/papers/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.pdf.

ConvNeXt

➤ ConvNeXt performance



Liu et al. "A ConvNet for the 2020s" CVPR 2022. https://openaccess.thecvf.com/content/CVPR2022/papers/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.pdf.

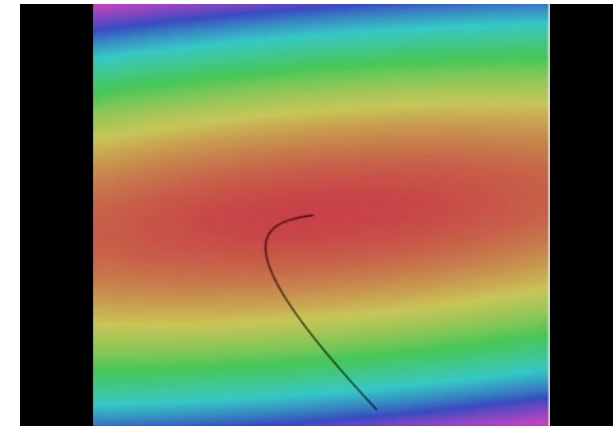
Training CNNs/Deep Neural Networks

CNN: Training

- A **loss function** is used to **compute the model's prediction accuracy from the outputs**
 - Commonly used: categorical cross-entropy loss function

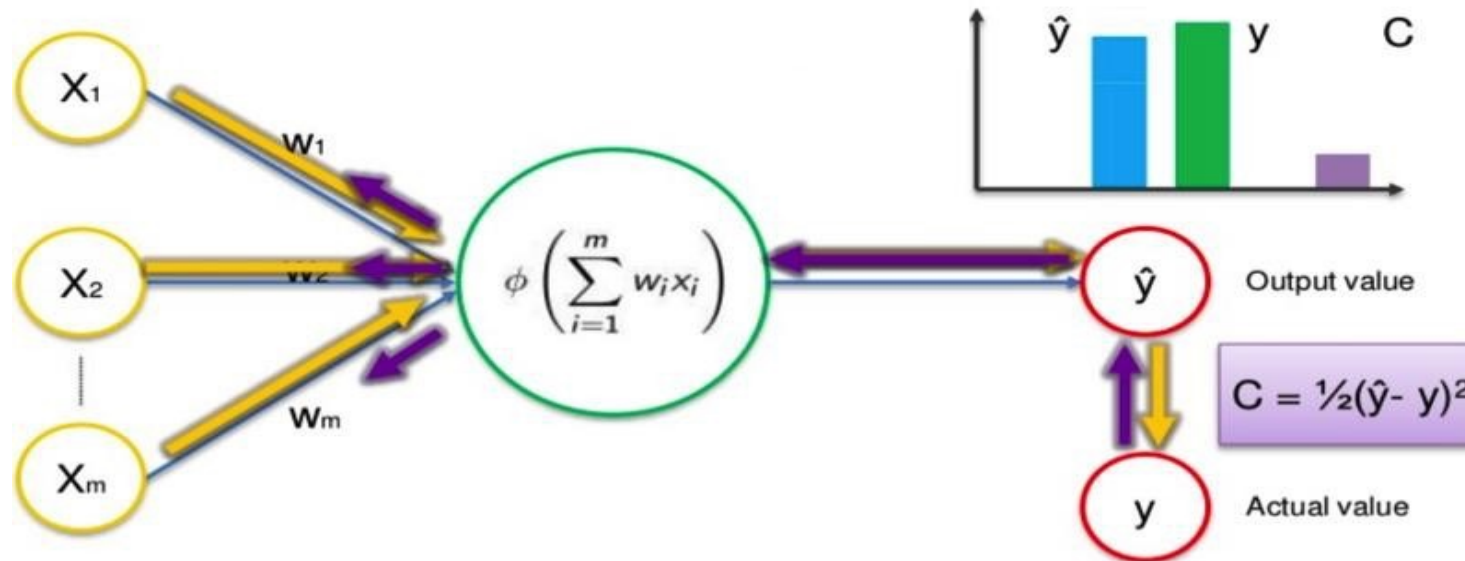
$$H(y, \hat{y}) = \sum_i y_i \log \frac{1}{\hat{y}_i} = - \sum_i y_i \log \hat{y}_i$$

- The training objective is to **minimise this loss**
- The **loss guides the backpropagation process to train the CNN model**
- Gradient descent based methods, such as Stochastic gradient descent and the Adam optimizer, are commonly used algorithms for optimisation



CNN: Training

- Backpropagation in general:
 - Calculating gradients for gradient descent
 - Directly deriving and calculating gradient is difficult, due to the complexity of DNNs



CNN: Training – PyTorch example

✓ Autograd in Training

We've had a brief look at how autograd works, but how does it look when it's used for its intended purpose? Let's examine how it changes after a single training batch. First, define a few constants, our model, and

```
[ ] BATCH_SIZE = 16
    DIM_IN = 1000
    HIDDEN_SIZE = 100
    DIM_OUT = 10

    class TinyModel(torch.nn.Module):

        def __init__(self):
            super(TinyModel, self).__init__()

            self.layer1 = torch.nn.Linear(DIM_IN, HIDDEN_SIZE)
            self.relu = torch.nn.ReLU()
            self.layer2 = torch.nn.Linear(HIDDEN_SIZE, DIM_OUT)

        def forward(self, x):
            x = self.layer1(x)
            x = self.relu(x)
            x = self.layer2(x)
            return x

    some_input = torch.randn(BATCH_SIZE, DIM_IN, requires_grad=False)
    ideal_output = torch.randn(BATCH_SIZE, DIM_OUT, requires_grad=False)

    model = TinyModel()
```

```
[ ] optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

    prediction = model(some_input)

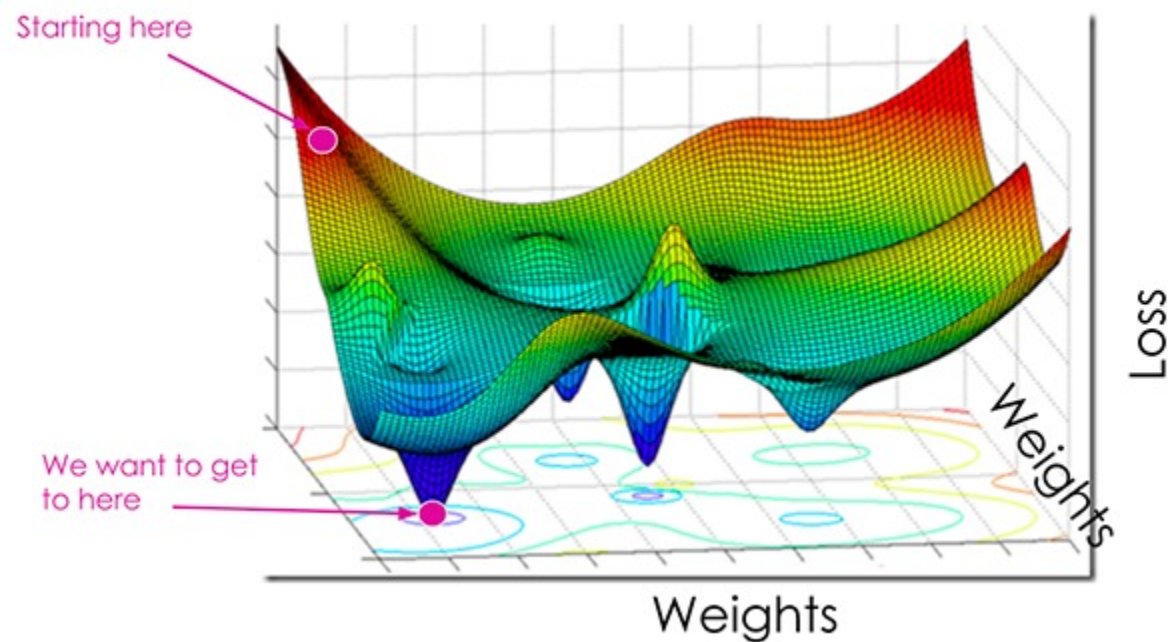
    loss = (ideal_output - prediction).pow(2).sum()
    print(loss)
```

```
[ ] loss.backward()
    print(model.layer2.weight[0][0:10])
    print(model.layer2.weight.grad[0][0:10])
```

```
[ ] optimizer.step()
```

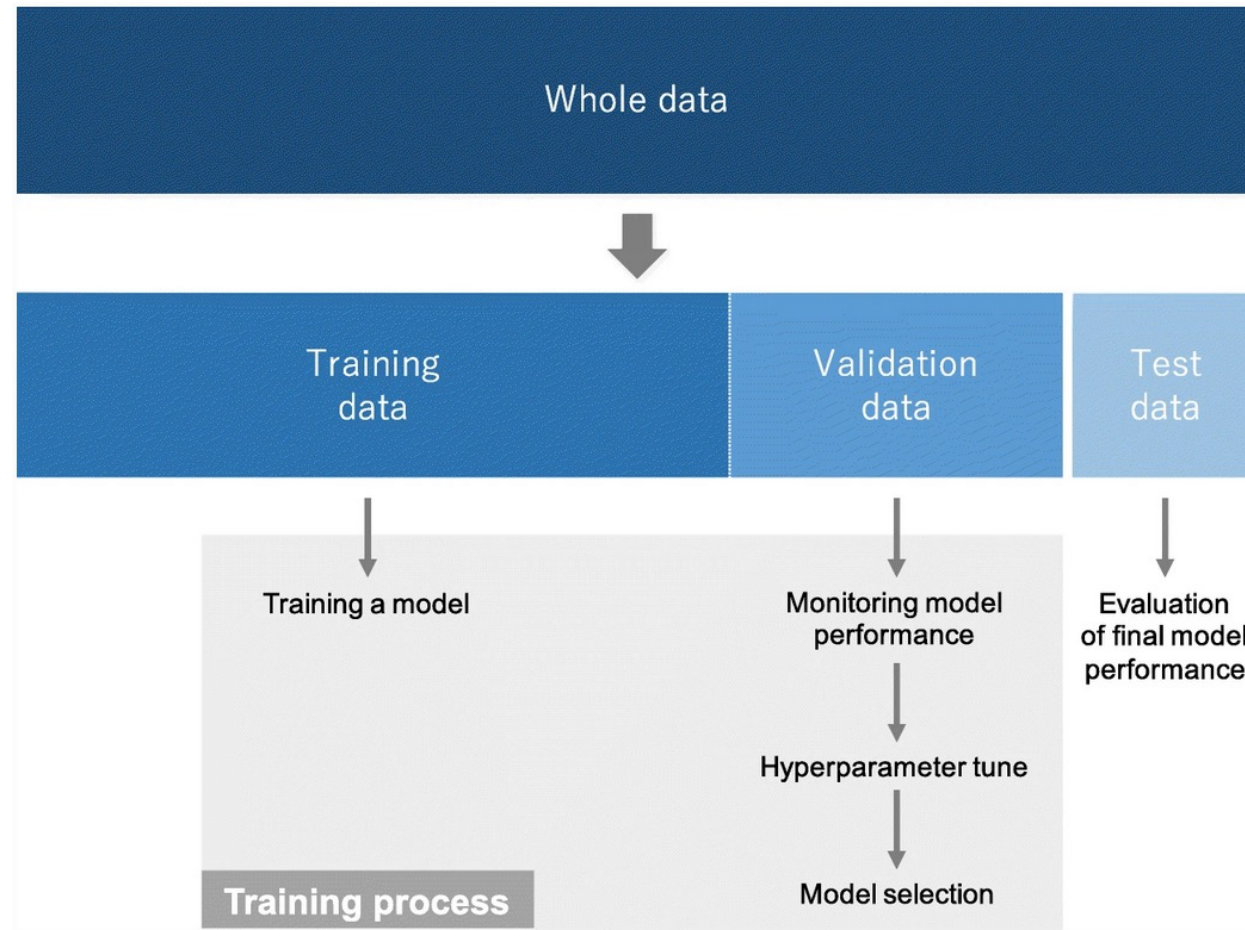
https://colab.research.google.com/github/pytorch/tutorials/blob/gh-pages/_downloads/ed9d4f94afb79f7dada6742a06c486a5/autogradyt_tutorial.ipynb

Why training Deep Neural Networks is hard?



Credit: Adrian Rosebrock, PyImageSearch, <https://www.pyimagesearch.com/2019/10/14/why-is-my-validation-loss-lower-than-my-training-loss/>

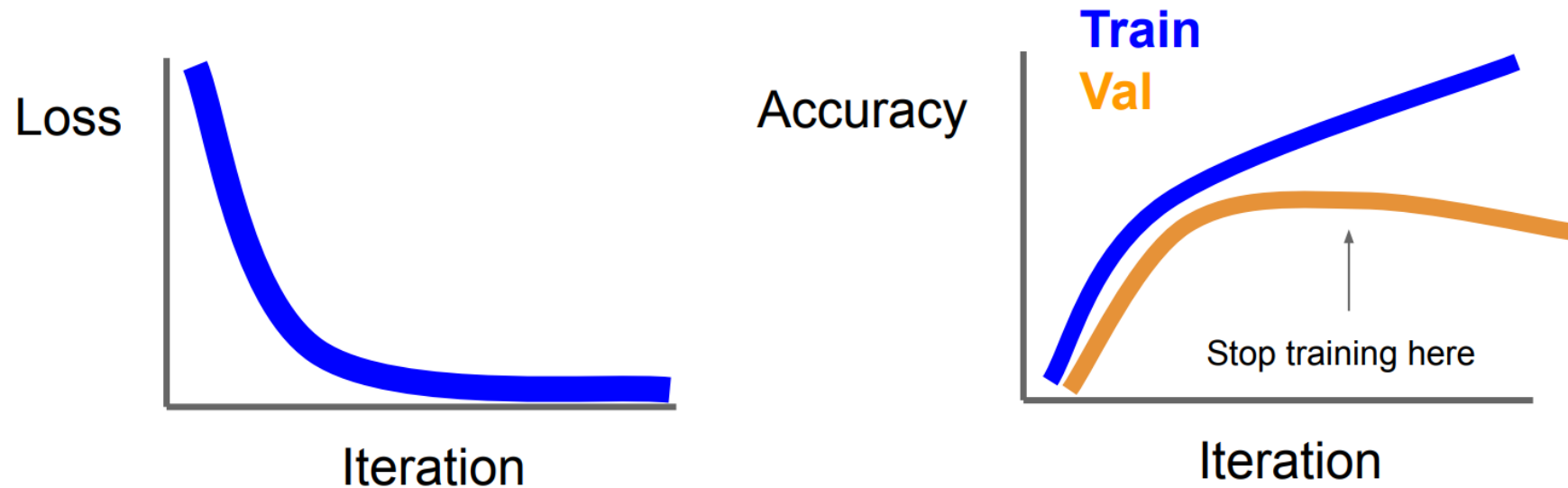
Training Methodology



Source: Yamashita R. et al. (2018) Convolutional neural networks: an overview and applications in radiology

Training vs. Testing Error

- Proper optimizer and training strategy can minimize the loss.
- Small training error is not always corresponding to a small testing/validation error.



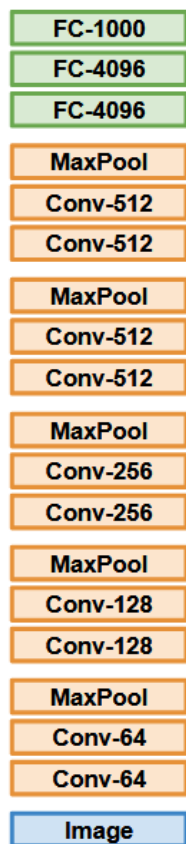
Transfer Learning

- Transfer learning aims to leverage the learned knowledge from a resource-rich domain/task to help learning a task with not sufficient training data.
 - Sometimes referred as **domain adaptation**
- The resource-rich domain is known as the **source** and the low-resource task is known as the **target**.
- Transfer learning works the best if the model features learned from the source task are **general** (i.e., domain-independent)

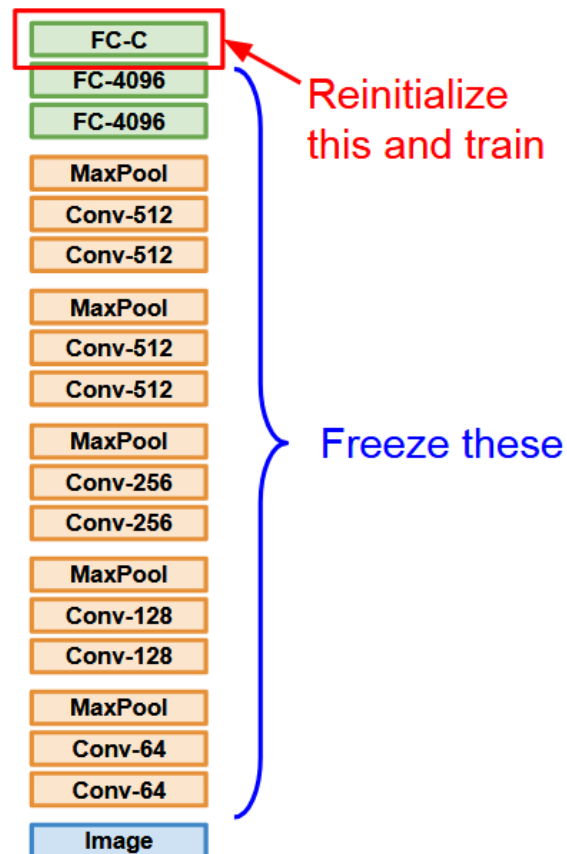
Credit: Mahammadreza Ebrahimi An Introduction to Deep Transfer Learning

Transfer Learning with CNNs

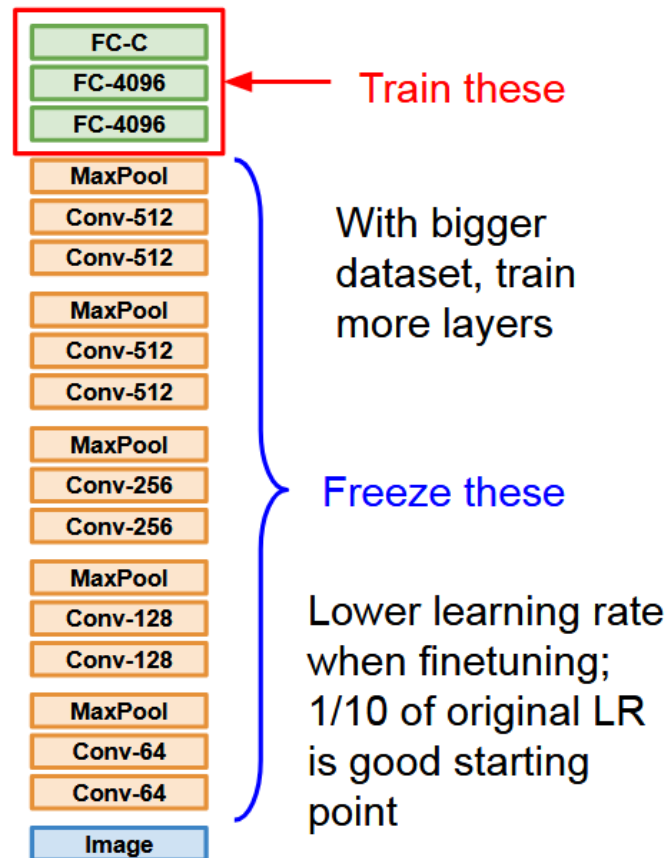
1. Train on Imagenet



2. Small Dataset (C classes)



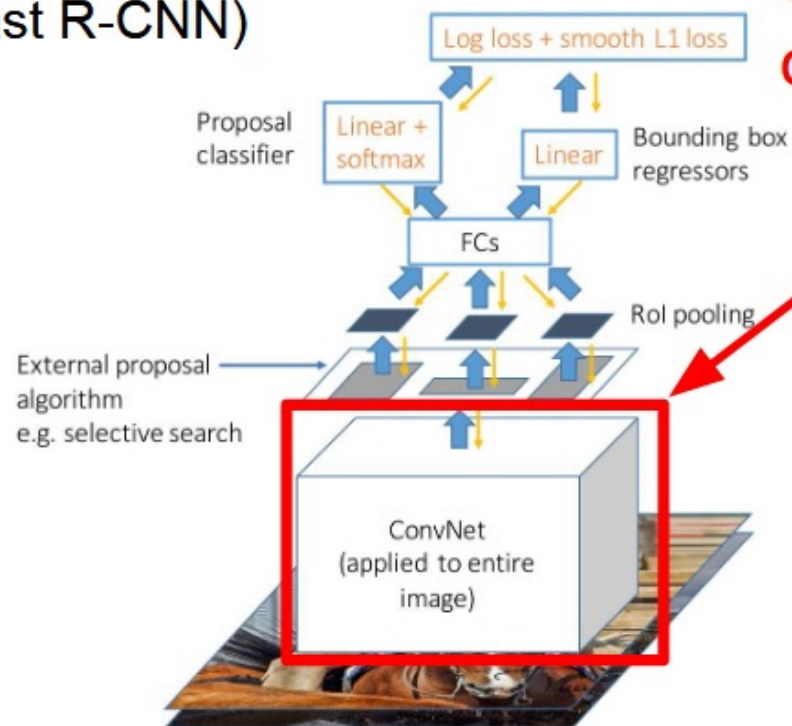
3. Bigger dataset



Slide Credit: Stanford CS231n Course

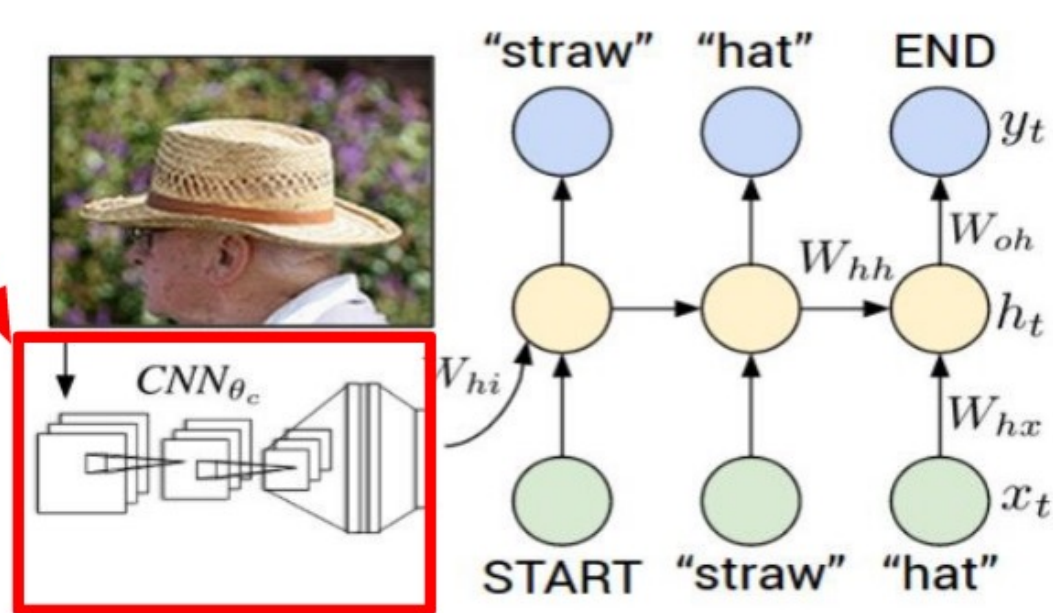
Transfer Learning is common in all applications

Object Detection (Fast R-CNN)



**CNN pretrained
on ImageNet**

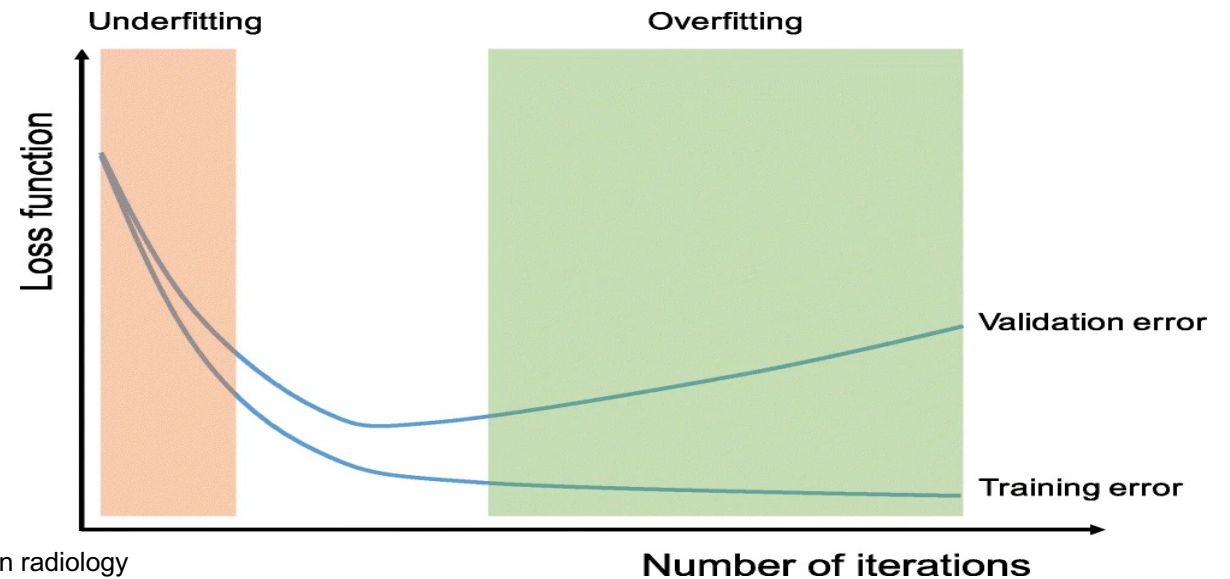
Image Captioning: CNN + RNN



Slide Credit: Stanford CS231n Course

Overfitting and Underfitting

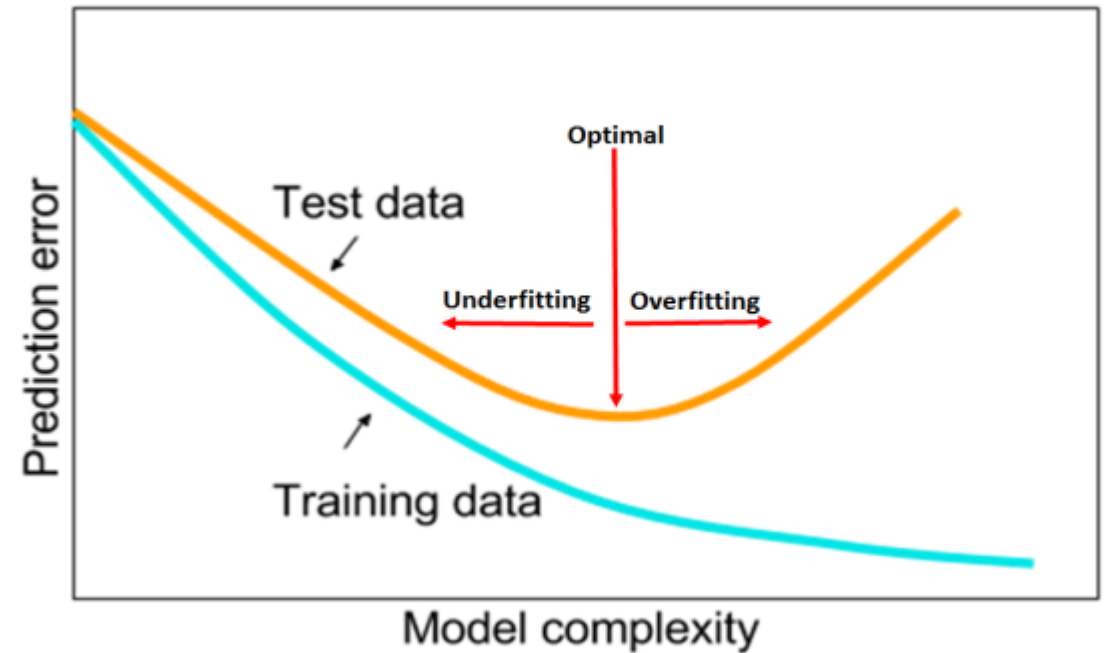
- Monitor the loss on training and validation sets during the training iteration.
- If the model performs poorly on both training and validation sets: Underfitting
- If the model performs well on the training set compared to the validation set: Overfitting



Source: Yamashita R. et al. (2018) Convolutional neural networks: an overview and applications in radiology

Common methods to mitigate overfitting

- More training data
- Early Stopping
- Data Augmentation
- Regularization (weight decay, dropout)
- Batch normalization



Source: Yamashita R. et al. (2018) Convolutional neural networks: an overview and applications in radiology
Image Credit: Hyper-parameters tuning practices: learning rate, batch size, momentum, and weight decay. Medium

More training data

- Costly
- Time consuming
- Need experts for specialized domains



Source: Fast Annotation Net: A framework for active learning in 2018. <https://medium.com/diffgram/fast-annotation-net-a-framework-for-active-learning-in-2018-1c75d6b4af92>
Image Datasets — ImageNet, PASCAL, TinyImage, ESP and LabelMe — what do they offer ? Medium Blog

Early Stopping

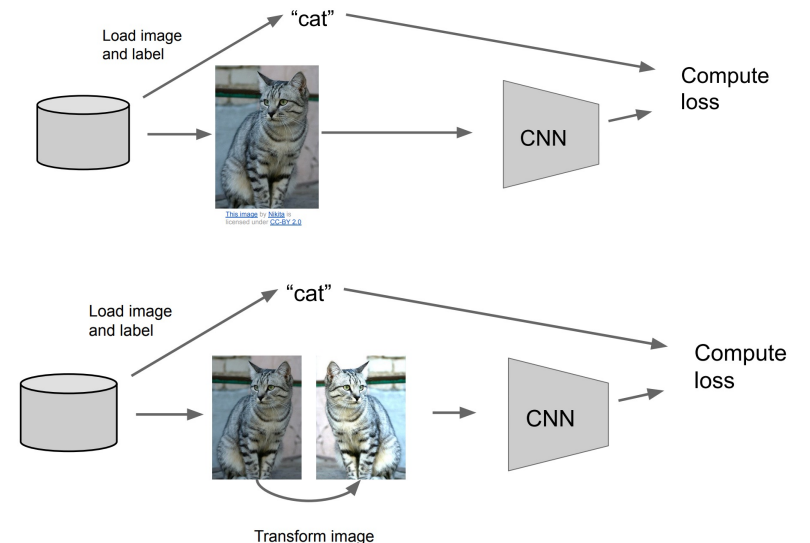
- Training too little mean model will underfit on the training and testing sets
- Training too much mean model will overfit the training dataset and hence poor performance on test set
- Early Stopping:
 - To stop training at the point when performance on a validation set starts to degrade.
 - Idea is to stop training when generalization error increases
- How to use Early Stopping
 - Monitoring model performance: Using metric to evaluate to monitor performance of the model during training
 - Trigger to stop training:
 - No change in metric over a given number of epochs
 - A decrease in performance observed over a number of epochs
 - Some delay or “patience” is good for early stopping

Source: Machine Learning Mastery: A Gentle Introduction to Early Stopping to Avoid Overtraining Neural Networks URL: <https://machinelearningmastery.com/early-stopping-to-avoid-overtraining-neural-network-models/>

Data Augmentation

- Data augmentation generate different versions of a real dataset artificially to increase its size
- Improving the robustness of the networks
- We use data augmentation to handle data scarcity and insufficient data diversity
- Data augmentation helps to increase performance of deep neural networks
- Common augmentation techniques:

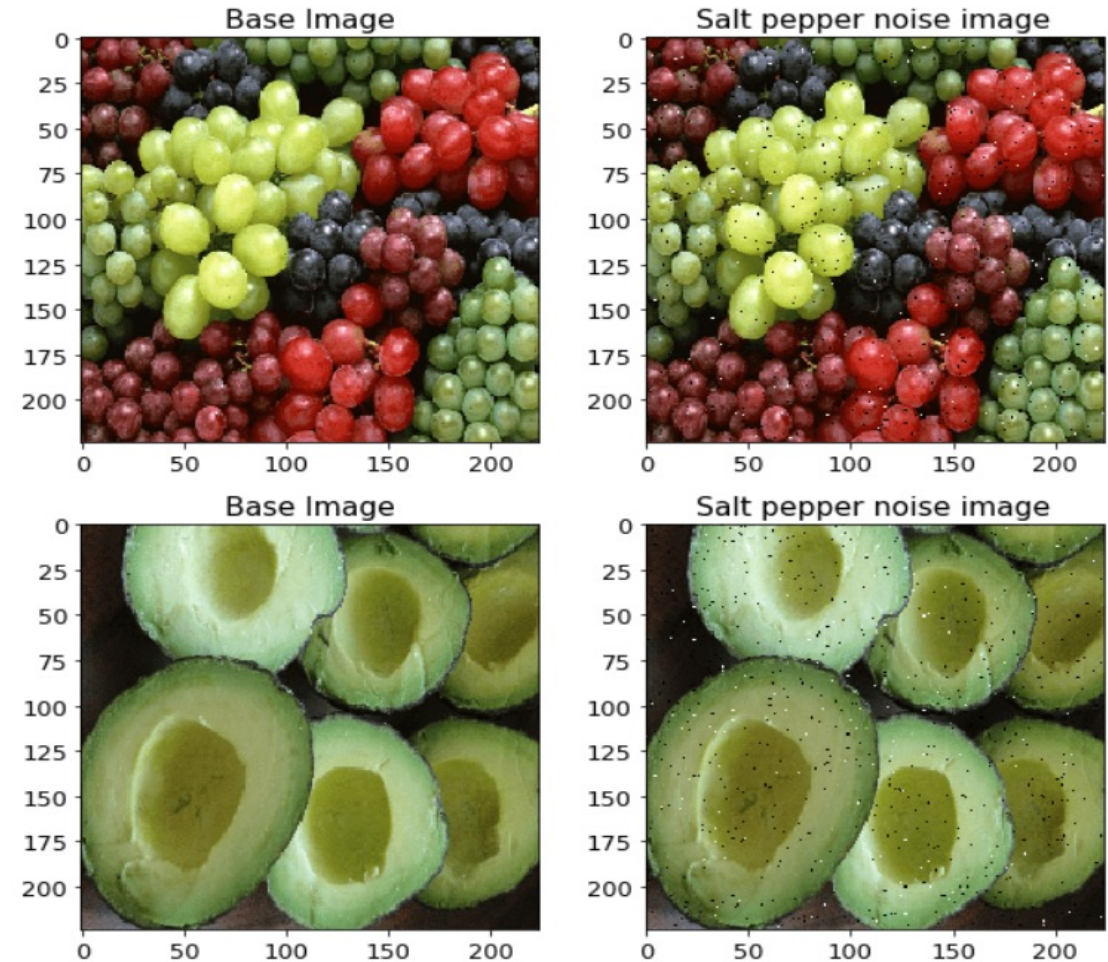
- Adding noise
- Cropping
- Flipping
- Rotation
- Scaling
- Translation
- Brightness
- Contrast
- Saturation
- Generative Adversarial Networks (GANs)



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

- Adding noise



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

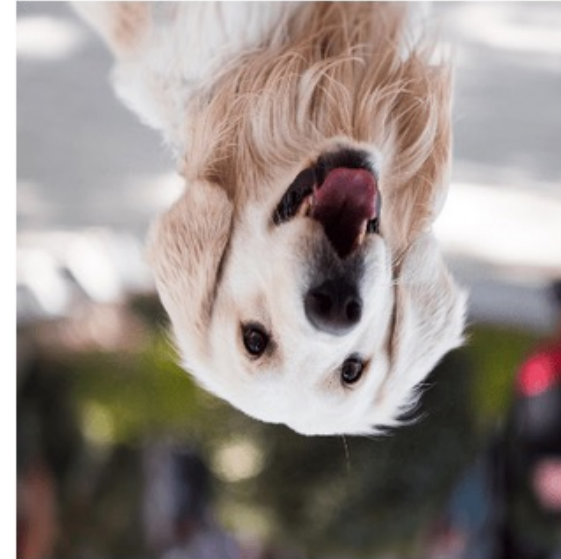
Data Augmentation

- Cropping



Data Augmentation

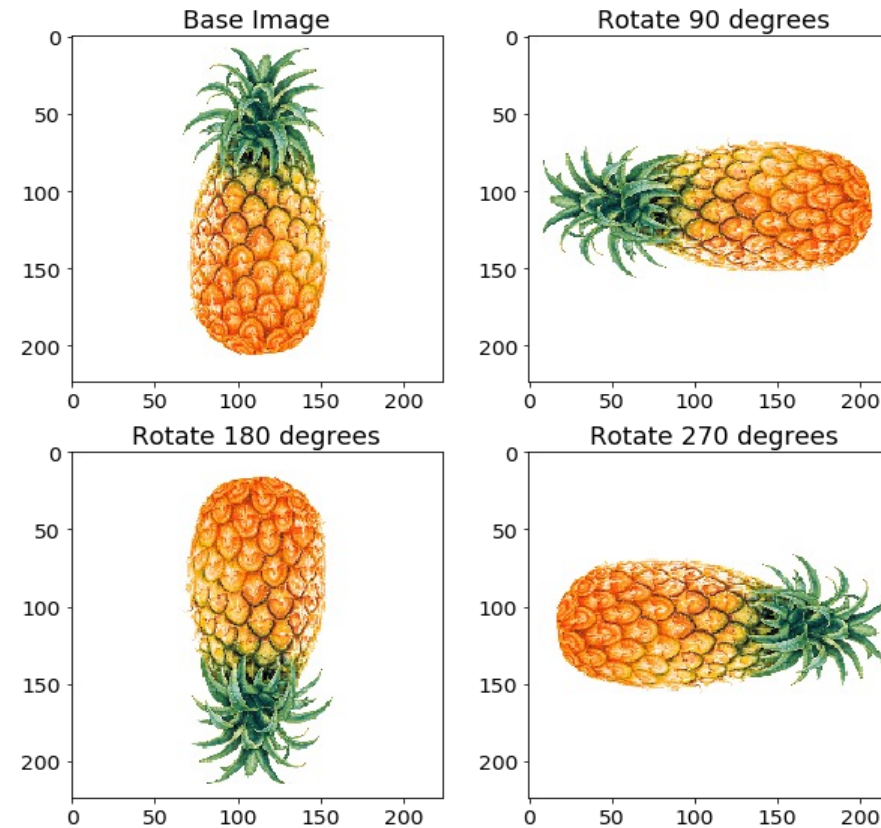
- Flipping



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

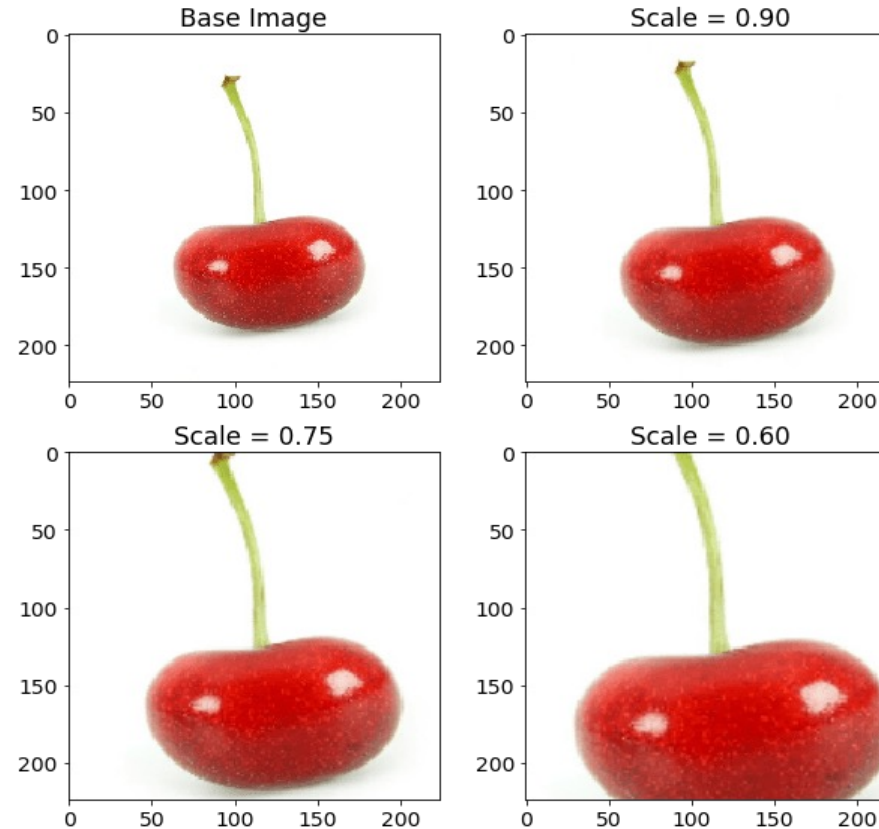
Rotation



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

- Scaling



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

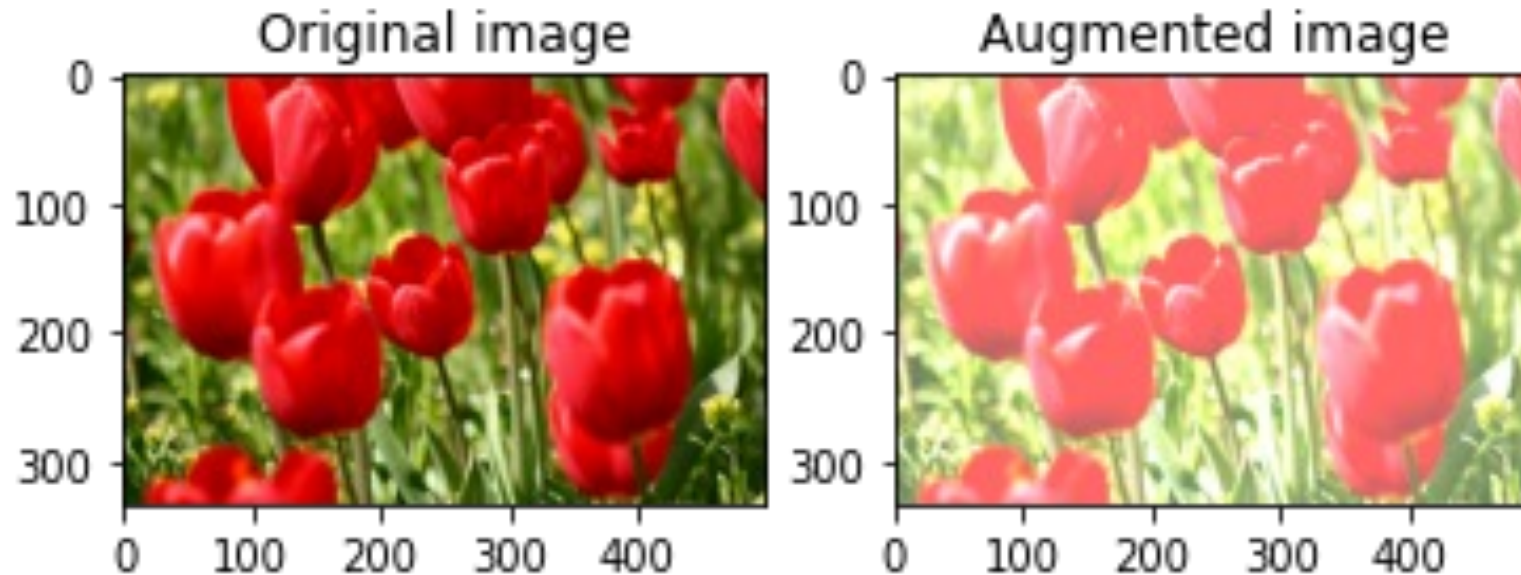
Translation



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>
<https://nanonets.com/blog/data-augmentation-how-to-use-deep-learning-when-you-have-limited-data-part-2/>

Data Augmentation

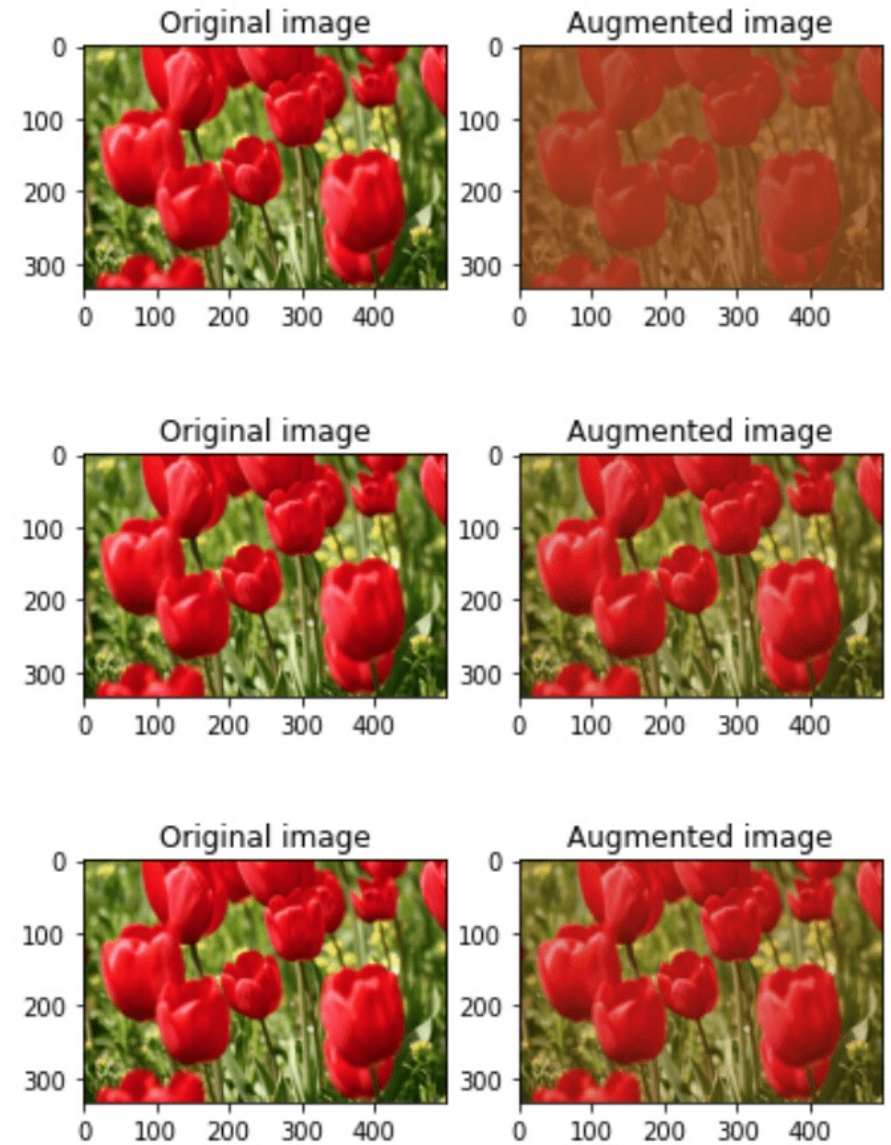
- Brightness



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

➤ Contrast



Source: 13 Data Augmentation Techniques. <https://research.aimultiple.com/data-augmentation-techniques/>

Data Augmentation

➤ Generative Adversarial Networks (GANs) for data augmentation



StyleGAN2 (baseline)

StyleGAN2 + DiffAugment (ours)

Source: Zhao et al., Differential Augmentation for Data-Efficient GAN Training, NeurIPS, 2020

Regularization: Weight Decay

- It adds a penalty term to the loss function on the training set to reduce the complexity of the learned model
- Popular choice for weight decay:
 - L1: The L1 penalty aims to minimize the absolute value of the weights

$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n |\theta_i|$$

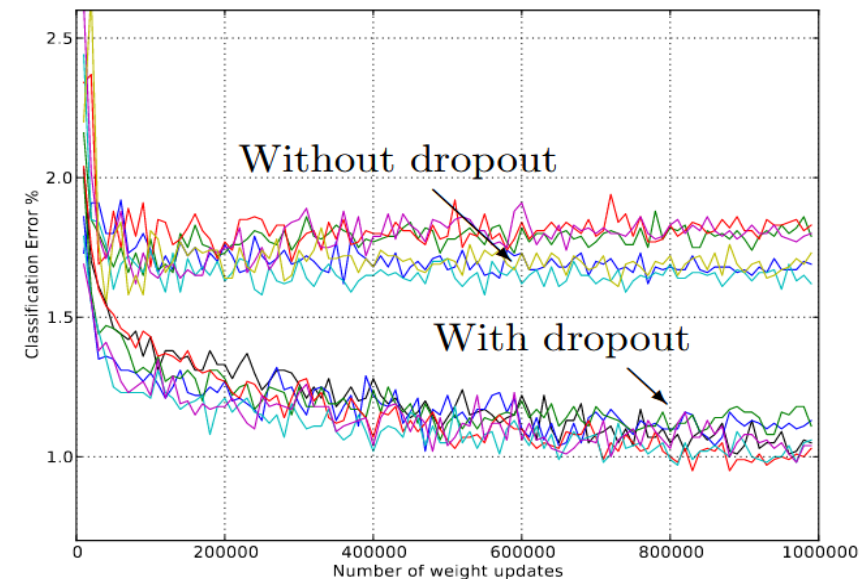
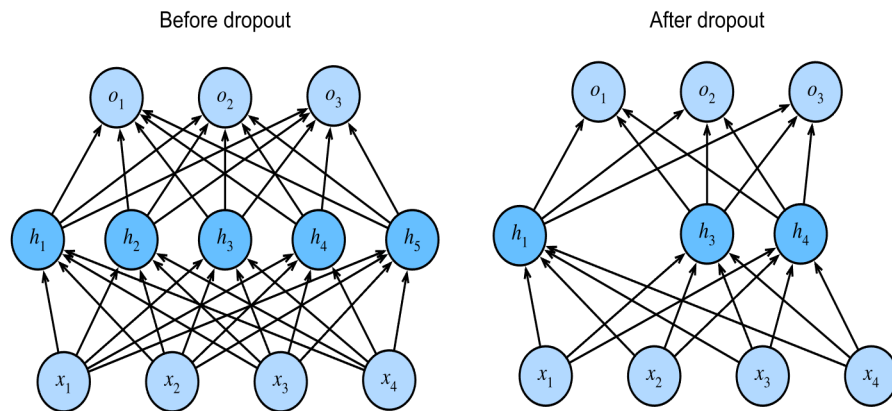
- L2: The L2 penalty aims to minimize the squared magnitude of the weights

$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n \theta_i^2$$

Credit: 5 Techniques to Prevent Overfitting in Neural Networks. <https://www.kdnuggets.com/2019/12/5-techniques-prevent-overfitting-neural-networks.html>

Regularization: Dropout

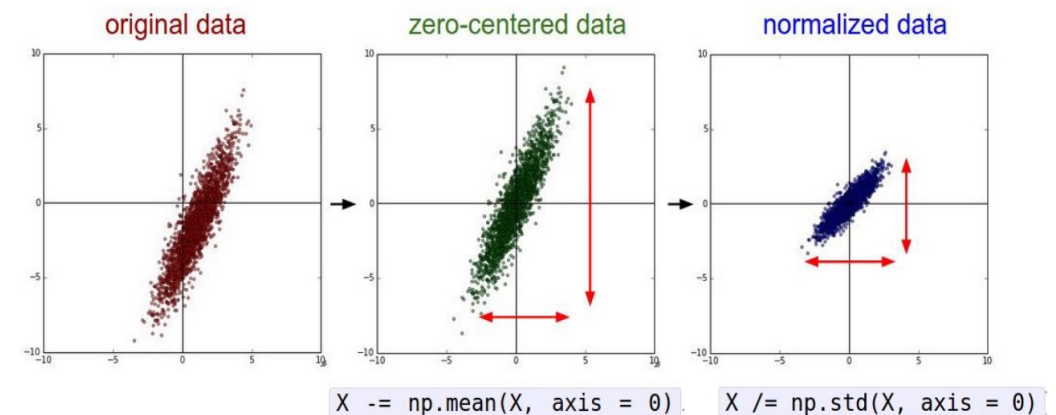
- L1 and L2 reduce overfitting by modifying the cost function
- Dropout regularizes the network by randomly dropping neurons from the neural network during training



Credit: Srivastava et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014
https://colab.research.google.com/github/d2l-ai/d2l-en-colab/blob/master/chapter_multilayer-perceptrons/dropout.ipynb

Data Preprocessing

- The pixel values in images must be scaled prior to given as input to deep neural networks for training or evaluation
- Three main types of pixel scaling:
 - **Pixel Normalization:** scale pixel values to the range 0-1
 - **Pixel Centering:** scale pixel values to have a zero mean
 - **Pixel Standardization:** scale pixel values to have a zero mean and unit variance

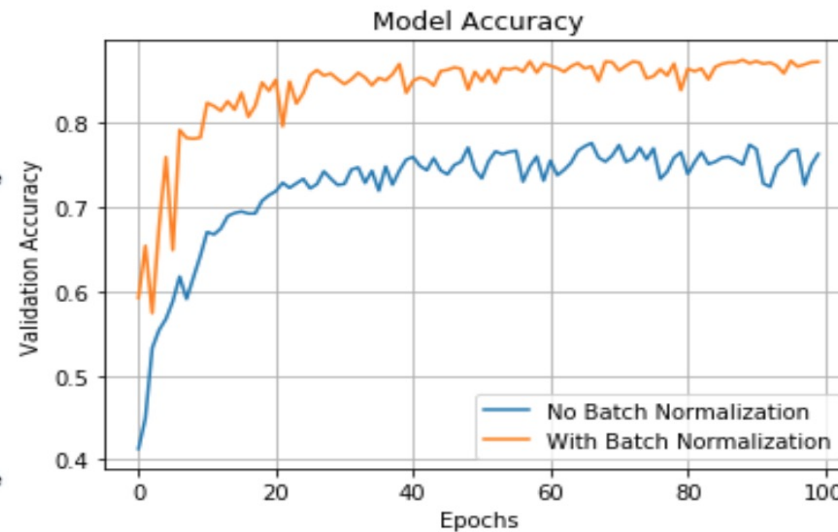
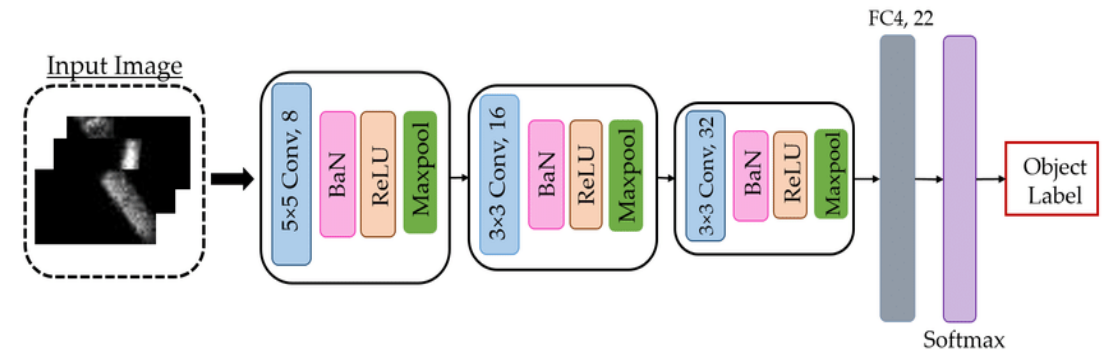


Credit: Stanford CS231n course slides.

Machine Learning Mastery: How to Normalize, Center, and Standardize Image Pixels in Keras

Batch Normalization

- Enables stable training
- Reduces the **internal covariate shift** (ICS)
- Accelerates the training process
- Reduces the dependence of gradients on the scale of the parameters



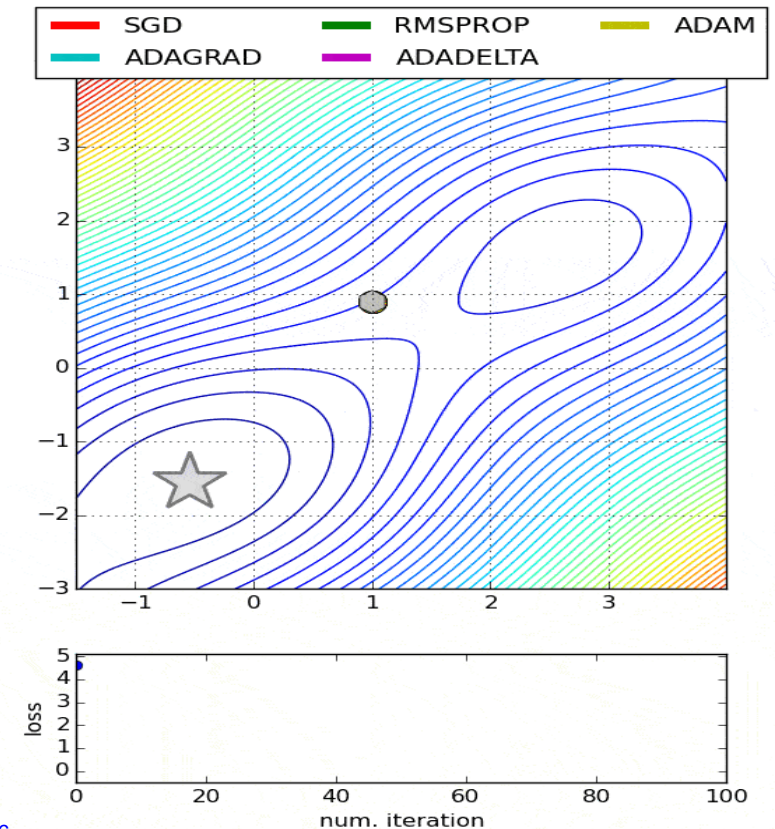
Source: LearnOpenCV: Batch Normalization in Deep Networks. <https://learnopencv.com/batch-normalization-in-deep-networks/>

Choice of Optimizers

➤ Choosing right optimizer helps to update the model parameters and reducing the loss in much less effort

➤ Most DL frameworks supports various optimizers:

- Stochastic Gradient Descent (SGD)
- Momentum
- Nesterov Accelerated Gradient
- AdaGrad
- AdaDelta
- Adam
- RMSProp



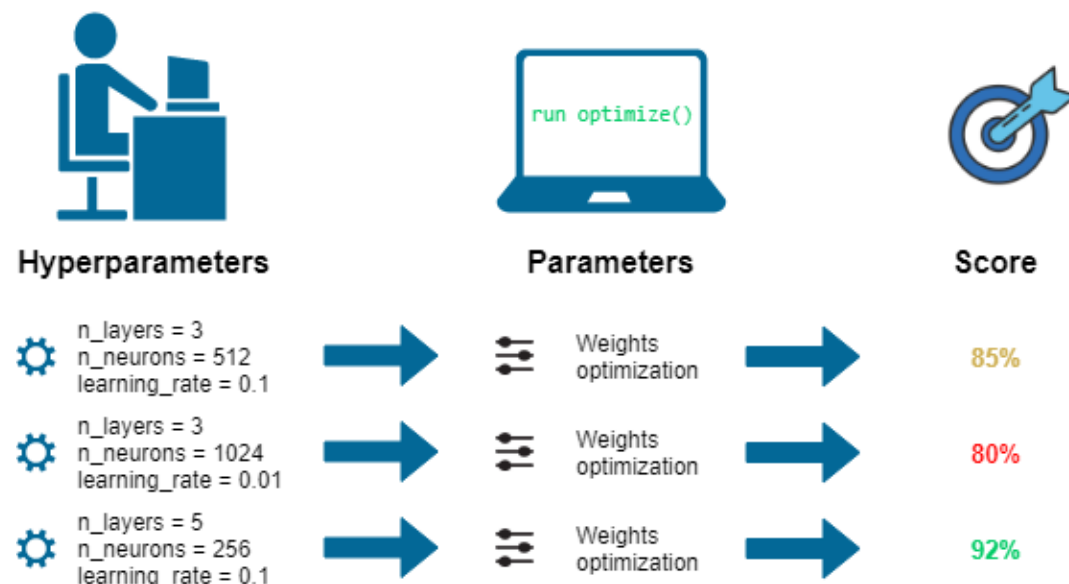
Source: Towards Data Science. Various Optimization Algorithms For Training Neural Network <https://towardsdatascience.com/optimizers-for-training-neural-network-59450d71caf6>

Tuning Hyperparameters

- Hyperparameters are all parameters which can be arbitrarily set by the user before starting training
- Hyperparameters are like knobs or dials of the network (model)
- An optimization problem: We aim to find the right combinations of their values which can help us to find either the minimum (e.g., loss) or the maximum (e.g., accuracy) of a function

- Many hyperparameters to tune:

- Learning rate
- No. of epochs
- Dropout rate
- Batch size
- No. of hidden layers and units
- Activation function
- Weight initialization
- ...



Source: KDnuggets: Practical Hyperparameter Optimization. <https://www.kdnuggets.com/2020/02/practical-hyperparameter-optimization.html>

Tuning Hyperparameters strategies

➤ Random Guess

- Simply use values from similar work

➤ Rely on your experience

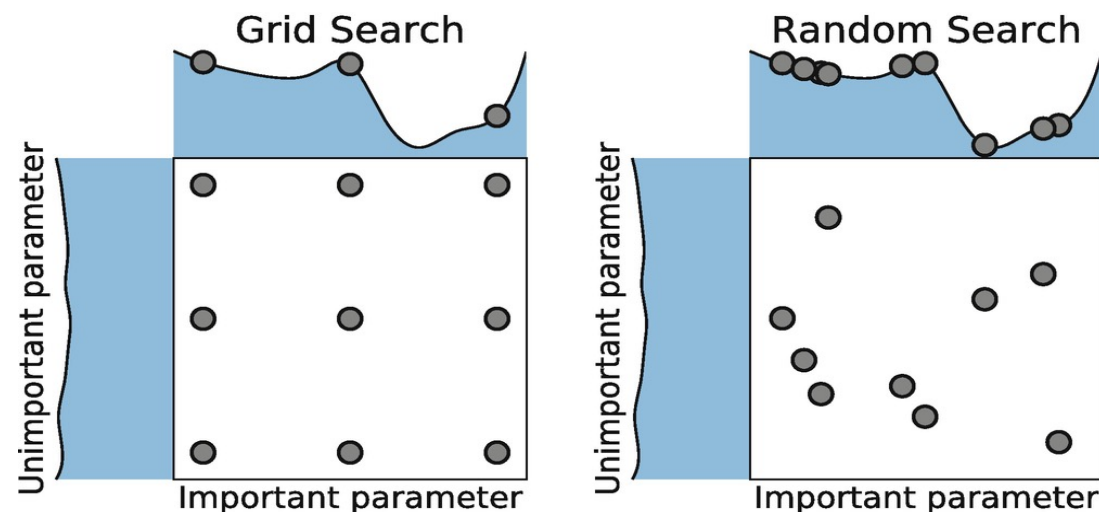
- Training DNNs is part art, part science
- With experience sense of what works and what doesn't
- Still chances of being incorrect (suboptimal performance)

➤ Grid Search

- Set up a grid of hyperparameters and train/test model on each of the possible combinations

➤ Automated hyperparameter tuning

- Use of Bayesian optimization and Evolutionary Algorithms
- Hyperopt: Distributed Asynchronous Hyperparameter Optimization



Deep Learning Frameworks/Packages

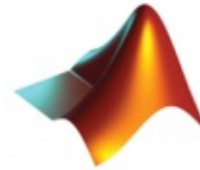
Caffe



Chainer



Microsoft
Cognitive
Toolkit



MATLAB



 PaddlePaddle

 PyTorch


TensorFlow



 Keras

Recent trends in Deep Learning

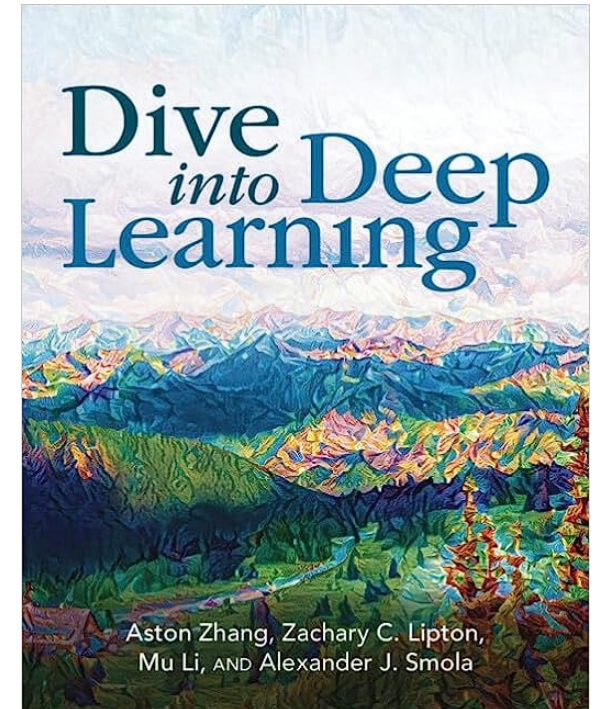
- Unsupervised Learning
- Self-supervised Learning
- Transformers and LLMs: The Game Changers
- Explainable AI (XAI): Demystifying the Black Box
- Federated Learning: Privacy-Preserving AI
- Edge AI: Bringing Intelligence to the Source
- Neuromorphic Computing: Mimicking the human brain
- AI Ethics and Governance: Building Responsible AI

<https://www.linkedin.com/pulse/future-deep-learning-trends-emerging-technologies-devfiinc-rxzec/>

Demo Examples

Demo Example 1

- AlexNet – PyTorch
 - https://d2l.ai/chapter_convolutional-modern/alexnet.html
- VGG16
 - https://d2l.ai/chapter_convolutional-modern/vgg.html
- ResNet
 - https://d2l.ai/chapter_convolutional-modern/resnet.html
 - https://colab.research.google.com/github/d2l-ai/d2l-pytorch-colab/blob/master/chapter_convolutional-modern/resnet.ipynb



<https://d2l.ai/index.html>

Key takeaways

➤ Training methodology

- Split data into training (such as 70%), validation (10%), and testing (20%)
- Take care of data leakage
- Check distribution of classes, work on balanced datasets (ideally)
- Find and develop baseline models at first
- Tune hyperparameters on validation set. Save best model and do inference on test set (once)
- Don't use off-the-shelf model blindly. Do ablation studies to know its working

➤ Data augmentation techniques are not standardized

- Get input from experts to know what data augmentations make sense in the domain

➤ Results

- Use multiple metrics rather a single metric to report results (often they are complementary)
- Show both qualitative and quantitative results (e.g., image segmentation)

Acknowledgements

- Slides from
 - <https://syncedreview.com/2020/06/23/google-deepmind-researchers-revamp-imagenet/>
 - Lecun et al. (1989). Gradient-based learning applied to document recognition.
 - Deep Learning's Most Important Ideas. <https://www.kdnuggets.com/2020/09/deep-learnings-most-important-ideas.html>
 - <https://medium.com/coinmonks/paper-review-of-vggnet-1st-runner-up-of-ilsvlc-2014-image-classification-d02355543a11>
 - Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).
 - Source: Convolutional Neural Networks. <https://medium.com/@rajat.k.91/convolutional-neural-networks-why-what-and-how-f8f6dbebb2f9>
 - Image Credit: Medium. https://medium.com/@pierre_guillou/understand-how-works-resnet-without-talking-about-residual-64698f157e0c
 - Source: Convolutional Neural Networks. <https://medium.com/@rajat.k.91/convolutional-neural-networks-why-what-and-how-f8f6dbebb2f9>

Example exam question

What does transfer learning with CNNs involve?

- A. Training a model from scratch for each new task.
- B. Using a pretrained model and fine-tuning it for a new task.
- C. Converting image data to text data to facilitate learning.
- D. Combining multiple CNNs into a single model for better performance.